

DE-44 (EE)

HAZIQ ZIA, M. USMAN MASOOD, AHSAN SHAUKAT, ABDUL RAFAY

MEDICAL DELIVERY UGV



**COLLEGE OF
ELECTRICAL AND MECHANICAL ENGINEERING NATIONAL
UNIVERSITY OF SCIENCES AND TECHNOLOGY RAWALPINDI
2026**



DE-44 EE PROJECT REPORT

MEDICAL DELIVERY UGV

Submitted to the Department of Electrical Engineering in partial fulfillment of the
requirements for the degree of
Bachelor of Engineering in
Electrical 2026

Supervisor:

Submitted By:

Dr Taosif Iqbal

Haziq Zia(415264)
M. Usman Masood-(423585)
Ahsan Shaukat-(432406)
Abdul Rafay Malik-(406107)

Acknowledgements

This Final Year Project thesis could not have been successful without various people who directly and indirectly involved themselves in this project work. To begin with, we express our heartfelt appreciation to our supervisor, Dr. Taosif Iqbal, who was always there to assist us with continuous feedback while working on the project. His knowledge of robotics and control system influences our research direction and technical approach.

We are extremely thankful to **Department of Electrical Engineering, College of Electrical and Mechanical Engineering, National University of Sciences and Technology Rawalpindi** for providing the laboratory infrastructure, computing facilities and educational ambience for performing research and development work in a professional way.

A special thanks to our families for their unwavering support, patience and encouragement throughout the more trying times of this project. We were able to overcome a number of technical challenges because they believed we could do it.

Our classmates and seniors who were well versed in embedded systems, power electronics, robotic navigation aided us, also deserve a thank you. I found their helpfulness in collaborating with debugging and testing truly commendable.

We owe special thanks to the companies and open-source communities of the Raspberry Pi Foundation, BreezySLAM, and multiple Python libraries, which became the software backbone of our self-driving car. It would not have been possible in the time while we would have been allowed to do an academic project without their free tools and documentation.

Finally, we thank the external reviewers and examination committee for their time and effort in evaluating our work. We remain grateful for this opportunity to contribute to the field of autonomous robotics and power electronics.

Abstract

The report describes the design, implementation and validation of an Autonomous Unmanned Ground Vehicle (UGV) that is capable of force analysis, stable power management and real-time navigation indoors without the Robot Operating System (ROS). This project aims to cater to the growing demand for standalone robotic platforms that are low cost for applications such as hospital material delivery, warehouse automation, and indoor surveillance.

The UGV is engineered by various engineering disciplines. A simulator to assess force interaction was implemented to quantitatively analyse the traction and torque requirements for traversal of the inclined surface to select a suitable motor and battery. The power management subsystem is designed around a synchronous buck converter that is customised using the MSMP2307DN-LF-Z controller that provides a 5 V, 5 A output for an input of 9-36 V. The type-II compensation was designed analytically and verified with the help of MATLAB based Bode analysis. The phase margin was established as 49.5 degree with crossover frequency at 3.2 kHz. This establishes guaranteed loop stability at all loading conditions.

The UGV's python software stack used for autonomous Navigation runs on a Raspberry Pi 5. Wheel odometry is taken from Hall effect encoders (47 ticks per revolution) with differential kinematics. An MPU6050 provides extra yaw rate feedback. A discrete time PID heading controller with $K_p=0.03$, $K_i=0$ and $K_d=0.01$, was tuned and the stability of the controller was confirmed through closed loop step-response and root-locus analysis using MATLAB without the Control System Toolbox. The BreezySLAM library that makes SLAM (Simultaneous localization and mapping) possible is used in processing 360-degree LiDAR scans from an RPLidar A1 to an occupancy grid map. The A* search algorithm is utilized to conduct global path planning on the generated grid, with waypoints being sequentially followed employing encoder-based dead reckoning.

The RealVNC server connectivity makes possible remote monitoring and manual teleoperation compatible with Raspberry Pi 5 on X11 and Wayland display servers. Each subsystem was tested in isolation and in integration and verified. Subsequently, an end-to-end test proved successful autonomously traversing from a start location to a destination while avoiding obstacles. This project relates to SDG of Industry, Innovation and Infrastructure and Sustainable Cities and Communities (SDG 11) is to help make these affordable, available and open-source Robotics Technology on of Service Automation.

Sustainable Development Goals

The proposed project will directly contribute to the achievement of relevant United Nations Sustainable Development Goals (SDGs), namely SDG 9 and SDG 11 through the development of an accessible, open-source autonomous ground vehicle platform.

SDG 9: Industry, Innovation, and Infrastructure

“Resilient infrastructure, inclusive and sustainable industrialization and innovation” are required for the end of economic development. This project presents a low-cost autonomous robotic UGV developed with low-cost consumer sensors (RPLidar A1, MPU6050) and open-source software libraries that acts as a working robotic platform. The project plans to refrain from using middlewares like ROS while implementing all algorithms in pure Python to lower the entry barrier for robotics research in developing countries. The innovation pertains to a buck converter with stable margins outputting 5V, 5A designed specifically for mobile robotics with energy-efficient power supply system. The custom-designed buck converter with strict stability proof and MATLAB verification further illustrates a systematic approach to power electronics design that can be adapted and continued to other embedded systems. Another item in the list is that the assembly of low-cost commercial-off-the-shelf components (Raspberry Pi 5, RPLidar A1, MPU6050, JGA25-370 motors) into an autonomous robot platform shows that commercial hardware can be assembled to make a low-cost platform for industrial and service applications.



SDG 11: Sustainable Cities and Communities

The aim of SDG 11 is to create an inclusive, safe, resilient, and sustainable space. The hospital medical delivery use case which motivates this UGV addresses a key urban logistics problem: the timely medical supply transport within the hospital complex. The UGV reduces human dependence on repetitive tasks like transport by automating intra-facility delivery, thus allowing medical staff to focus on patients. It is especially true during health crises or in understaffed facilities as resource optimization is vital.

The obstacle avoidance and autonomous navigation guarantee safe operation in humans. AGV systems are usually reliant on physical infrastructure (magnetic tape, QR-code markers, etc.). However, this UGV uses onboard perception (LiDAR) and localization (encoder + gyro SLAM) and can thus be deployed in existing buildings without expensive retrofits. By extending the application of existing facilities, the sustainability of urban infrastructure can be facilitated without appealing for new construction. The lithium battery chemistry selected for the motor

drive has a higher energy density and a longer cycle life than conventional NiCd alternatives, thereby reducing material waste during the vehicle lifetime.

In conclusion, through a combination of rigorous engineering analysis, an open-source implementation, and a socially relevant application domain, this project shows how electrical engineering research can make a contribution to sustainable development.



List of Symbols

Symbol	Description
--------	-------------

Latin Letters

A	Acceleration (m/s^2)
C	Capacitance (F); also wheel circumference (m)
d	Distance traveled (m)
E	Error signal in control system
$e[k]$	Discrete-time error at sample k
F	Force (N)
F_g	Gravitational force (N)
F_N	Normal force (N)
$F_{g\perp}$	Perpendicular component of gravity (N)
$F_{g\parallel}$	Parallel component of gravity (N)
F_t	Traction force (N)
f_c	Crossover frequency (Hz)
f_s	Switching frequency (Hz)
f_z	Compensator zero frequency (Hz)
$G_c(s)$	Compensator transfer function
$G_p(s)$	Plant transfer function
g	Acceleration due to gravity, 9.81 m/s^2
I	Current (A); also integral term in PID
K_d	Derivative gain
K_i	Integral gain
K_p	Proportional gain
L	Inductance (H); also wheelbase (m)
m	Mass (kg)
N	Number of samples or ticks
P	Proportional term; also power (W)
R	Resistance (Ω); also wheel radius (m)
s	Laplace variable
T	Sampling period (s); also torque (Nm)
$T(s)$	Open-loop transfer function
t	Time (s)
$u[k]$	Control signal at sample k
V	Voltage (V)
V_{in}	Input voltage (V)
V_{out}	Output voltage (V)
V_{ref}	Reference voltage (V)
v	Velocity (m/s)
X, Y	Cartesian coordinates (m)
x, y	Position coordinates (m)

Greek Letters

Symbol	Description
α	Angular acceleration (rad/s ²)
Δt	Time step (s)
$\Delta\theta$	Change in heading (rad)
θ	Heading angle, inclination angle (rad or °)
μ	Coefficient of friction
τ	Motor time constant (s); also torque (Nm)
ω	Angular velocity (rad/s)
ω_z	Gyroscope z-axis rate (°/s)
<i>Acronyms</i>	
ADC	Analog-to-Digital Converter
AGV	Automated Guided Vehicle
AMCL	Adaptive Monte Carlo Localization
BEC	Battery Eliminator Circuit
CSV	Comma-Separated Values
DC	Direct Current
EKF	Extended Kalman Filter
ESR	Equivalent Series Resistance
FYP	Final Year Project
GPIO	General-Purpose Input/Output
H-Bridge	Half-Bridge motor driver topology
I2C	Inter-Integrated Circuit
IMU	Inertial Measurement Unit
LiDAR	Light Detection and Ranging
MCU	Microcontroller Unit
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
MPU	Microprocessor Unit
NiCd	Nickel-Cadmium
OMML	Office Math Markup Language
PCB	Printed Circuit Board
PID	Proportional-Integral-Derivative
PM	Phase Margin (°)
PWM	Pulse-Width Modulation
RMS	Root Mean Square
ROS	Robot Operating System
SDG	Sustainable Development Goal
SLAM	Simultaneous Localization and Mapping
SSH	Secure Shell
TPR	Ticks Per Revolution
UART	Universal Asynchronous Receiver-Transmitter
UGV	Unmanned Ground Vehicle
USB	Universal Serial Bus
VNC	Virtual Network Computing

TABLE OF CONTENT

Acknowledgements	3
Abstract	4
Sustainable Development Goals.....	5
List of Symbols.....	7
Introduction	12
Background	12
Problem Statement	13
Objectives	13
Report Structure	15
Background and Literature Review	16
Forces on an Inclined Surface	16
Buck Converter Fundamentals	17
Principle of Operation.....	17
Current-Mode Control	18
Feedback Network and Output Voltage Setting.....	18
Type-II Compensation	19
Stability Metrics: Phase Margin and Crossover Frequency.....	19
Odometry and Differential Kinematics	20
Encoder Calibration.....	21
Gyroscope Integration and Drift	21
PID Control Theory	22
Continuous-Time Formulation.....	22
Discrete-Time Implementation	22
Simultaneous Localization and Mapping	23
Occupancy Grid Mapping.....	23
Scan Matching.....	24
A* Path Planning	25
Communication Protocols	25
Pulse-Width Modulation and H-Bridge Motor Control.....	26
RC Filter Design for Sensor Noise Reduction.....	26
Raspberry Pi 5 Architecture	26
2.8 RPLidar A1 Sensor Specifications	27
2.9 MPU6050 Inertial Measurement Unit	28
2.10 Literature Review Summary	28
Methodology.....	29
Force Simulator.....	29
Physics Model and Equations	29
User Interface Design	30
Buck Converter Design	31
IC Selection and Specifications	31

Feedback Resistor Divider.....	32
Type-II Compensation Network.....	32
Transfer Function Derivation.....	33
MATLAB Verification Without Toolbox.....	34
Scaling and Visualization Logic.....	35
Odometry and PID Control.....	35
Encoder Interface and Calibration.....	35
Differential Kinematics Implementation.....	36
Discrete PID Heading Controller.....	36
Motor Trim Calibration.....	38
MATLAB Step Response and Root Locus.....	38
PWM Motor Control Theory.....	39
Low-Pass Filtering of Sensor Data.....	40
SLAM and Mapping.....	40
Data Logging Architecture.....	40
A* Path Planning.....	41
Grid-Based A* Implementation.....	41
Waypoint Following.....	42
Hardware Assembly and Wiring.....	43
1.1 Software Architecture and Multithreading.....	44
Safety Features and Emergency Stop.....	46
Results.....	48
Force Simulator Outputs.....	48
Buck Converter Stability.....	48
PID Performance.....	49
Simulation Results.....	49
Root-Locus Analysis.....	49
Physical Driving Tests.....	49
SLAM Mapping.....	50
Autonomous Navigation.....	50
Performance Characterization.....	50
VNC Connection.....	51
Conclusions and Future Work.....	52
Summary of Achievements.....	52
Lessons Learned.....	52
Future Work.....	53
REFERENCES.....	55
MATLAB Code for Buck Converter Bode Plot.....	56
Appendix B.....	58
Appendix C.....	61
APPENDIX D — UGV TUNING MANUAL FOR CREW MEMBERS.....	63

Overall Power: The Speed Limit.....	63
Manual Trim: Fixing Natural Drift.....	63
The PID Controller: Auto-Steering	63
C.1 Standard Tuning Sequence.....	64

Introduction

Background

In the last 20 years, mobile robotics has advanced remarkably due to the parallel development of sensors, embedded computers and machine learning algorithms. Unmanned Ground Vehicles (UGVs) are among the most versatile classes of mobile robots, capable of operating in an indoor structured environment or an outdoor environment with various degrees of difficulty-level. A UGV is a robotic vehicle which works on the surface of ground without a human operator on board. They work on pre-programmed instructions, remote teleoperated and with full autonomous degree.

The universal aspiration to hand over dangerous, repetitive or onerous tasks to machines is the underlying reason for developing autonomous UGVs. In the military, UGVs are used for explosive ordinance disposal, reconnaissance, and logistics to protect humans from harm. Autonomous ground vehicles are increasingly used in warehouse automation, farming, elderly care, and hospital logistics for civilian missions. The pandemic of COVID19 showed the importance of contactless delivery. As a result, researchers started the deployment of service robots in healthcare institutions.

The project mentioned in this report is aimed at the hospital delivery use case. The movement of drugs, specimens, documents, and equipment between wards, pharmacies, and laboratories in a hospital setting is a critical logistical function. However, this movement takes up an inordinately high amount of time of nurses and other support staff. In next-generation autonomous UGVs which could travel in corridors, call elevators, and deliver material to floor level, medical attendants would be freed from running around the hospitals ferrying materials.

Unfortunately, building an entirely autonomous UGV from the ground up – i.e. without the use of high-level middleware frameworks, such as the Robot Operating System (ROS) – presents enormous engineering difficulties. The robot must solve at the same time a number of problems, namely perception (understanding where it is), localization (knowing where it is), mapping (creating a map of where it is), planning (deciding to go where), and control (getting there). Advanced mathematics, sensors, and software engineering are required for each of these domains.

Besides navigational intelligence, designing any UGV requires force analysis and power management too. In a case where a vehicle is travelling up an inclined ramp or an uneven floor, the vehicle should generate enough traction force to overcome the gravitational pull in the backward direction. Without a force analysis of the robot, it will have no other way but to guess the needed motor. The result will be either insufficient torque leading to failure to climb slopes, or over-selection of motor mass which leads to overspending. The power supplies for the various onboard electronics microprocessor, sensors, motor drivers, communication modules need to be regulated and be stable. Input not detected. Please provide input text.

This project therefore employs a holistic systems-engineering approach that integrates mechanical, electrical, and software autonomy into one project component. The vehicle's main

differential-drive chassis has two DC gear motors with a hall effect encoder. The primary computer of the robot is a Raspberry Pi 5 running pure-Python software for motor control, encoder counts, IMU readings, LiDAR acquisition, SLAM, path planning and GUI rendering. The Raspberry Pi, along with the peripheral sensors that utilize it, require a stable 5 V rail together with the custom-designed buck converter on MSMP2307DN-LF-Z IC.

Problem Statement

The present solutions for indoor autonomous navigation tend to be either expensive commercial platforms that rely on proprietary ecosystems or open-source ROS-based stacks that require a lot of computing power and have a steep learning curve.

Undergraduate engineering projects and small research laboratories in resource-constrained environments face obstacles through both approaches.

1. A standalone autonomous navigation system is a system that can navigate without any external connection or input.
2. Runs completely independently of ROS or other heavyweight middleware systems
3. This system operates efficiently on a Raspberry Pi 5, RPLidar A1, and MPU6050.
4. Reliably navigates indoors using a combination of SLAM and A* path planning developed in pure Python.
5. The switching regulator was subjected to rigorous analysis and compensation to deliver power stably under all load conditions.
6. Quantifies and visualizes the mechanical forces acting on the vehicle when it traverses an incline to select motor and tune control parameters.

The second issue is the calibration and synchronization of heterogeneous sensors whose sampling rates, noise characteristics, and reference frames differ significantly. This includes the wheel encoders, gyroscope, and LiDAR. The wheel encoder operates at great frequency but drifts due to slip. The MPU6050 gyroscope measures rotation over small time intervals but introduces bias errors during integration. The LiDAR produces a lot of geometric information about the environment but has a low scan rate (approximately RPLidar A1 at 7 Hz). This project will achieve the tight integration of software in order to achieve a coherent estimate of robot pose and map structure from the disparate data, rather than through algorithmic filtering.

Objectives

The project objectives have been organized into 6 major deliverable, which is each subsystem of the autonomous UGV.

Objective 1: The objective of the first simulator is to create a graphical force-analysis simulator. The simulator will take in user-adjustable parameters like vehicle mass, wheel radius, and slope angle. Then, the graphical force-analysis simulator will compute and show

the gravitational decomposition, normal force, parallel drag force, required traction force, and wheel torque.

The simulator shall proportionally scale My force vector arrows and indicate the minimum coefficient of friction to prevent wheel slip. This tool gives instant quantitative feedback in the mechanical design stage.

Objective 2: The goal is to design an up-down (buck) converter that can deliver 5 V at a current of 5 A. The converter should work for a 9 V to 36 V input. It can be used to power Raspberry Pi 5 and the sensors. The power can be taken from a 2S or 3S LiPo battery. Also, stable voltage at the output should be maintained.

The design must include full feedback network calculations, Type-II compensation component selection, derivation of plant and compensator transfer functions, MATLAB verification of loop stability via Bode plot analysis (no Control System Toolbox). The aim is to have a phase margin of more than 45 degrees.

Objective 3: Verifying with MATLAB without using toolbox Write your own MATLAB code using only basic mathematical functions i.e. do not use tf, pid, margin, rlocus etc.

- Choose word or phrases from the below to create new 10 words
- Determine the crossover frequency and phase margin of 0dB.
- Draw the Bode magnitude and phase curves.
- By continuously varying the proportional gain K_p and resolving the characteristic equation at every value, the root locus is graphically plotted.
- The objective here is to ensure that a fundamental understanding of stability is achieved rather than black-boxed.

Objective 4: Odometry and PID Control To implement wheel odometry using Hall effect encoders (47 ticks per revolution; 0.038 m wheel radius) and differential kinematics to keep a track of the robot position and heading. Design a discrete-time PID controller for maintaining heading while driving in a straight line, with parameter tuning for disturbance rejection due to an uneven floor. Generate MATLAB step-response and root-locus plots without using a toolbox for the tuned controller.

Objective 5: Rasterisation and Navigation A* on Raspberry Pi 5 Use the BreezySLAM library to implement Simultaneous Localization and Mapping that uses scans from the RPLidar A1 and encoder odometry to create an occupancy grid map of the environment. Use the A* path-planning algorithm for collision-free path generation on the grid created. Implement an automatic waypoint follow strategy by combining gyroscopic heading control with encoder-based travelling distance measurement.

Objective 6: Remote VNC Access Set the RealVNC server on the Raspberry Pi 5 for remote desktop connectivity of the Raspberry Pi 5 from the laptop on LAN. Put down documents on

the various configurations for X11 and Wayland, as well as troubleshooting options for common connection issues like port mismatch and display server mismatch.

Report Structure

The remaining sections of this report are organized as follows.

Chapter 2 discusses the theoretical foundations of the entire subsystem. This touches on force decomposition on inclined planes, buck converter operation and compensation, kinematics of differential drive, discrete-time PID control theory, SLAM algorithms, A* search, and remote access protocols.

Chapter 3 details the design and implementation of all subsystems. These encompass the equations and user interface for the force simulator complete buck converter schematic with components calculations encoder calibration and odometry maths PID discrete formulation and tuning methodology SLAM data-logging pipeline A* implementation on occupancy grids and VNC configuration steps.

The outcomes observed in this chapter are measured and simulated. The document contains a discussion of tabulated force values, stability margins of the Bode plot, PID step-response characteristics, occupancy grid maps generated, A* paths run, and VNC screenshots.

In Chapter 5, the project's accomplishments are summarised, followed by outlining the limitations of the current implementation and directions for future improvements including rolling-resistance modelling, AMCL-based localising refinement, dynamic obstacle avoidance and multi-floor navigation.

The References section provides all the works cited in IEEE format. The Appendices present the complete MATLAB source code for the buck converter Bode plot and PID step-response/root-locus simulations.

Background and Literature Review

Forces on an Inclined Surface

As the UGV travels along the slope, the weight force acting on the centre of mass needs to be resolved into components up the slope and perpendicular to the slope. This breakdown is vital in assessing the ability of the vehicle's drivetrain to produce enough torque to keep moving as well as the friction between wheels and ground in preventing the slip.

Imagine a UGV with a mass m lying on a plane, which is inclined at an angle θ to the horizontal. The weight attracts downward through the centre of mass vertically downward with magnitude.

$$F_g = mg$$

where $g \approx 9.81 \text{ m/s}^2$. It is handy to take the x-axis parallel to the incline (positive upwards) and the y-axis perpendicular to the incline (positive outward from the incline). The weight vector has to be resolved into two mutually perpendicular components in this frame.

The vehicle is pressed down on the ground through the F_N normal reaction of the surface by the component perpendicular to the plane. Its magnitude is

$$F_{g\perp} = mg\cos\theta$$

By Newton's third law, the normal force has equal magnitude:

$$F_N = mg\cos\theta$$

The component parallel to the plane acts down the slope and opposes upward motion. Its magnitude is

$$F_{g\parallel} = mg\sin\theta$$

To climb the slope with constant velocity the total traction force F_t delivered by the wheels must at least equal this parallel component (neglecting rolling resistance and aerodynamic drag which are small at the low speeds of indoor UGVs):

$$F_t = F_{g\parallel} = mg\sin\theta$$

The torque that is required at each one of the drive wheels will depend on the wheel radius r and the number of drive wheels sharing the load. A differential-drive robot has two wheels that propel independently, and the traction force is shared. If load sharing is equal, the torque per wheel is

$$\tau = \frac{F_t \cdot r}{2} = \frac{mgr\sin\theta}{2}$$

The likelihood of wheel slip imposes another limitation. At each wheel-ground contact patch, the maximum static friction force available is μF_N where μ is the coefficient of static friction. In order for the vehicle to ascend without slipping, the traction required must not exceed the total available friction:

$$F_t \leq \mu F_N \Rightarrow \mu_{\min} = \frac{F_t}{F_N} = \tan\theta$$

This result is very simple in form, but it shows that the minimum friction coefficient depends only on the slope angle and not on mass. Given $\theta = 30^\circ$, we have $\mu_{\min} = \tan 30^\circ \approx 0.577$, this means the surface must supply up to the value of $\mu_{\min}$. When dry, smooth hospital floors may provide $\mu \approx 0.3\text{--}0.5$ so that steep ramps can present a genuine operational risk which the force simulator helps quantify.

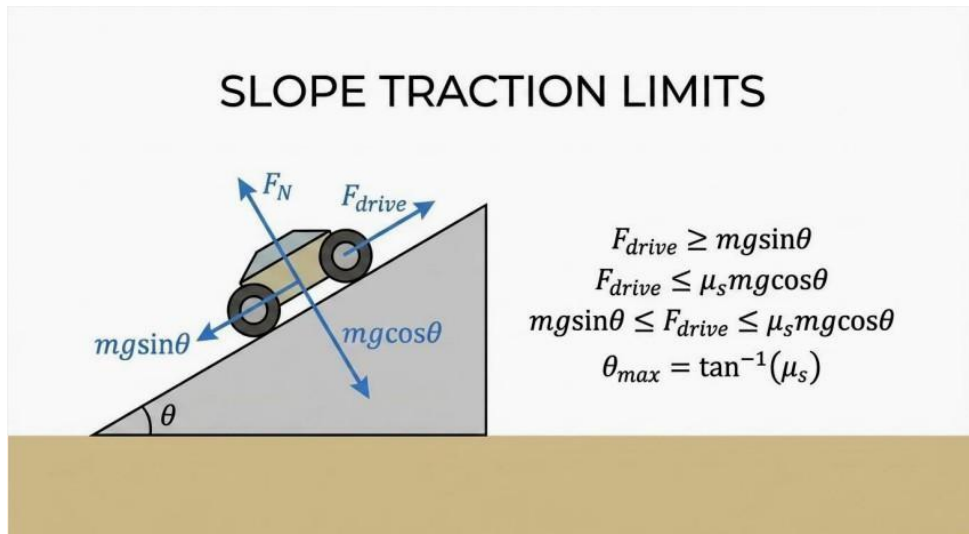


Figure 3: Free-body diagram of forces acting on a UGV traversing an inclined surface

Buck Converter Fundamentals

Principle of Operation

The buck converter is a switched-mode power supply (SMPS) that efficiently reduces a higher DC input voltage to a lower DC output voltage. It is also called a step-down converter. A buck regulator operates at a much higher efficiency than a linear regulator thanks to its high-frequency switching of a transistor on and off. Due to its on-state, an inductor stores energy, the power supply is maintained at a steady value.

The basic topology consists of four main components: a MOSFET switch that can be controlled (most common), a diode (or synchronous rectifier MOSFET), inductor, and output capacitor. Shortly after the switch closes the inductor current begins to rise. Thus the voltage developed across the inductor induces a rising magnetic field. Also, according to Faraday, the potential difference generated across inductor L at rate:

$$di_L/dt = (V_{in} - V_{out})/L$$

When the switch opens (off-time, t_{off}), the inductor's collapsing magnetic field maintains the current through the freewheeling diode, with $di_L/dt = -V_{out}/L$ (ideal diode drop). In steady state, the volt-second balance across the inductor requires the average voltage to be zero over one switching period $T_s = t_{on} + t_{off}$.

This gives the fundamental buck conversion ratio:

$$V_{out} = D \cdot V_{in}$$

where $D = t_{on}/T_s$ is the duty cycle, constrained to $0 < D < 1$.

For a switching frequency $f_s = 340$ kHz, the period is $T_s \approx 2.94 \mu s$. If the desired output is 5 V from a 12 V input, the required duty cycle is $D = 5/12 \approx 41.7\%$.

Current-Mode Control

The MSMP2307DN-LF-Z features peak current-mode control, where the inner loop regulates peak inductor current, instead of just output voltage. In current mode control the PWM comparator receives two inputs namely voltage-error-amplifier output (the control voltage V_c) and the sensed inductor-current signal (A current sense resistor or the on-resistance of the MOSFETs are generally used to obtain this signal). When the sensed current levels reach the limit which is set by V_c , it switches OFF. Accordingly, a fast inner current loop is created which is nested in the slower outer voltage loop.

Control using current has better benefits as compared to control using voltage. To begin with, the inductor gets removed effectively from the small-signal models of the power stage, due to the inner loop making it appear as a current-controlled voltage source. The plant order is reduced from two poles to one pole plus a high frequency zero, simplifying compensation design. Moreover, the MOSFET and inductor are safeguarded against overload by inherent cycle-by-cycle current limiting. In addition, the input voltage variations are immediately corrected by the current loop via the line voltage feedforward property. Thus, the voltage-error amplifier can respond much later.

Feedback Network and Output Voltage Setting

The output voltage of the circuit is determined by a resistive divider that connects to the error amplifier's inverting input. The MSMP2307 features an internal bandgap reference voltage of $V_{ref} = 0.925$ V. For the divider network with top resistor $R_{13}=43\Omega$ and bottom resistor $R_1=10\Omega$, the output voltage is.

$$V_{out} = V_{ref} \left(1 + \frac{R_{13}}{R_1}\right) = 0.925 \times \left(1 + \frac{43000}{10000}\right) = 0.925 \times 5.3 = 4.9025 \text{ V}$$

This is close enough to the desired nominal value of 5 V, given that resistors have a tolerance and there is trace resistance.

Type-II Compensation

It is required that the frequency response of the error amplifier is shaped and that the overall open-loop gain crosses zero dB with adequate phase margin for closed-loop stability. Since current-mode control produces a first-order plant for frequencies below half the switching frequency, a Type-II compensator (with one pole at origin, one zero, and one high-frequency pole) suffices.

A resistor $R_{12}=6.8\text{k}\Omega$ and capacitor $C_{12}=3.9\text{nF}$ connected between the COMP pin and ground forms the compensator network in series with the current out of the error amplifier. The compensator zero frequency is

$$f_z = \frac{1}{2\pi R_{12} C_{12}} = \frac{1}{2\pi \times 6800 \times 3.9 \times 10^{-9}} \approx 6.00 \text{ kHz}$$

This zero provides phase boost near the crossover frequency, counteracting the phase lag of the output capacitor's equivalent series resistance (ESR) zero and improving the phase margin.

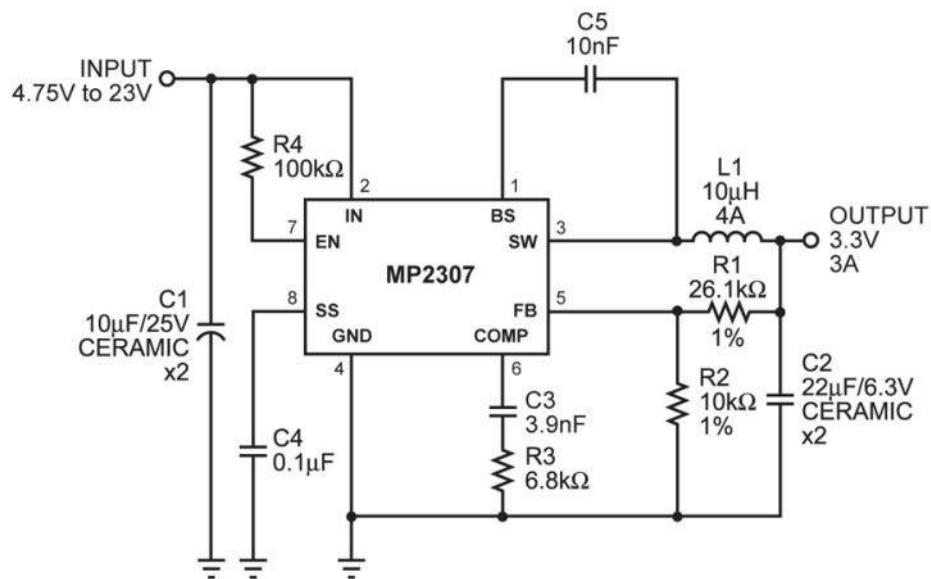


Figure 4: Synchronous buck converter schematic based on MSMP2307DN-LF-Z

Stability Metrics: Phase Margin and Crossover Frequency

Phase margin refers to the amount of excessive phase lag to achieve -180 degrees at the frequency when the open loop magnitude is unity (0 dB). A positive phase margin indicates stable negative feedback, while a margin under indicates positive feedback and oscillation. It is considered that for a good transient response the PM should be greater than 45 degrees and 60 degrees is assumed to be a golden value for a robust system that exhibits little overshoot.

The crossover frequency f_c (also called bandwidth or unity-gain frequency) indicates how fast the control loop can respond to disturbances. f_c (the voltage loop bandwidth) must be kept well below the switching frequency, usually $f_c < f_s/10$, to prevent the loop from trying to correct with a speed that the power stage can't switch. The suggested crossover frequency is nearly 34kHz for an f_s of 340kHz. The converter designed has $f_c = 3.2 \text{ kHz}$ and a considerable safety margin.

Odometry and Differential Kinematics

The way robot uses wheels to measure how rotation is successively estimated its position is called Odometry. The robot is inherently given an initial pose, which can be viewed as a form of dead reckoning, where the robot adds up small increments of displacement. It is true that odometry drifts due to wheel slip, uneven floors, encoder quantization, and other reasons. Nevertheless, odometry is important for short-term localization.

This is because it is updated at a high-frequency (hundreds of hertz) and does not require any infrastructure.

Let d_L and d_R be the incremental distances traveled by the left wheel and right wheel respectively in a small time interval Δt for a differential-drive robot having two independently actuated wheels separated by wheelbase L . The average amount is simply the arc length traversed by the robot's center point:

$$d_{center} = \frac{d_L + d_R}{2}$$

If $d_L \neq d_R$, the robot follows a circular arc of radius $R = (d_L + d_R)L/(2(d_R - d_L))$. The change in heading is

$$\Delta\theta = \frac{d_R - d_L}{L}$$

The new robot coordinates, assuming the motion occurred at the previous heading θ_{old} , are

$$\begin{aligned} x_{new} &= x_{old} + d_{center} \cos(\theta_{old}) \\ y_{new} &= y_{old} + d_{center} \sin(\theta_{old}) \\ \theta_{new} &= \theta_{old} + \Delta\theta \end{aligned}$$

The encoder update uses these equations calculated iteratively. Since the robot actually traces out a circular arc rather than straight line segments, the approximation is most accurate for

small step sizes. For slow-moving indoor robots, the error in linearization is not significant at normal loops rates above 20 Hz.

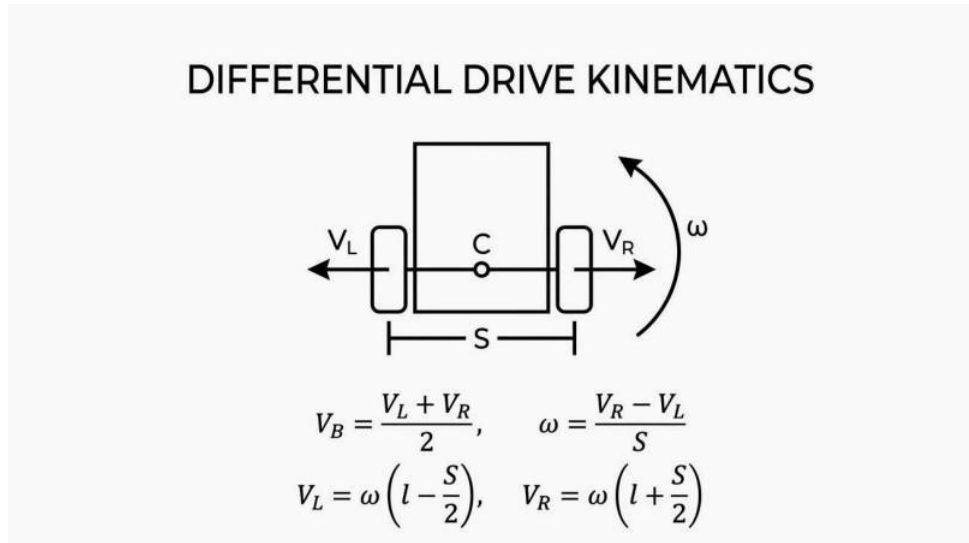


Figure 5: Differential-drive kinematic model showing wheel velocities and instantaneous centre of curvature

Encoder Calibration

The motors, JGA25-370 have the capacity of producing 47 ticks per revolution of the wheel with the help of Hall effect encoders. With a wheel radius $r = 0.038$ m, the wheel circumference is $C = 2\pi r \approx 0.2388$ m. The distance per encoder tick is

$$\Delta d = \frac{C}{\text{TPR}} = \frac{0.2388}{47} \approx 0.00508 \text{ m} = 5.08 \text{ mm}$$

There's approximately a 5 mm distance that each tick on the encoder disc represents. Such resolution is sufficient for indoors navigation, where waypoints can tolerate a few cm.

Gyroscope Integration and Drift

The MPU6050 inertial measurement unit contains a three-axis gyroscope that measures angular velocity ω_z around the vertical or the yaw axis. This rate, integrated over time, gives the heading change:

$$\Delta\theta = \int_{t_0}^{t_1} \omega_z(t) dt \approx \sum_k \omega_z[k] \cdot \Delta t$$

In the present project, gyroscope primarily used for short-term stabilization of heading in fast PID control loop, while slower LiDAR based SLAM corrects the long-term drift.

PID Control Theory

Continuous-Time Formulation

The most common algorithm used for a feedback controller is the Proportional-Integral-Derivative (PID) controller. The output $u(t)$ is a weighted sum of three terms acting on the error signal $e(t) = r(t) - y(t)$, where $r(t)$ is the reference (setpoint) and $y(t)$ is the measured process variable:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

The proportional term $K_p e(t)$ immediately responds, being proportional to the actual error. The integral term $K_i \int e(\tau) d$ is added to remove the steady state offset. The term $K_d \frac{de}{dt}$ indicates that the error's rate of change may cause future error causing little oscillation created.

Discrete-Time Implementation

Due to the fact that the Raspberry Pi completes the control loop at discrete intervals, continuous PID must be digitally approximated. At sample k , the output of the discrete-time PID, using the backward-difference approximation for the derivative and rectangular integration for the integral, is

$$u[k] = K_p e[k] + K_i T_s \sum_{j=0}^k e[j] + K_d \frac{e[k] - e[k-1]}{T_s}$$

where T_s is the sampling interval. To prevent integral windup – the condition where the integral term goes too large because of being held too long in saturation and causes large amounts of overshoot, the integral accumulator is clamped to finite values. This speed is sufficient for controlling the heading due to the motor-gearbox assembly's mechanical time constant. The fast loop runs at roughly 20 Hz ($T_s \approx 50$ ms).

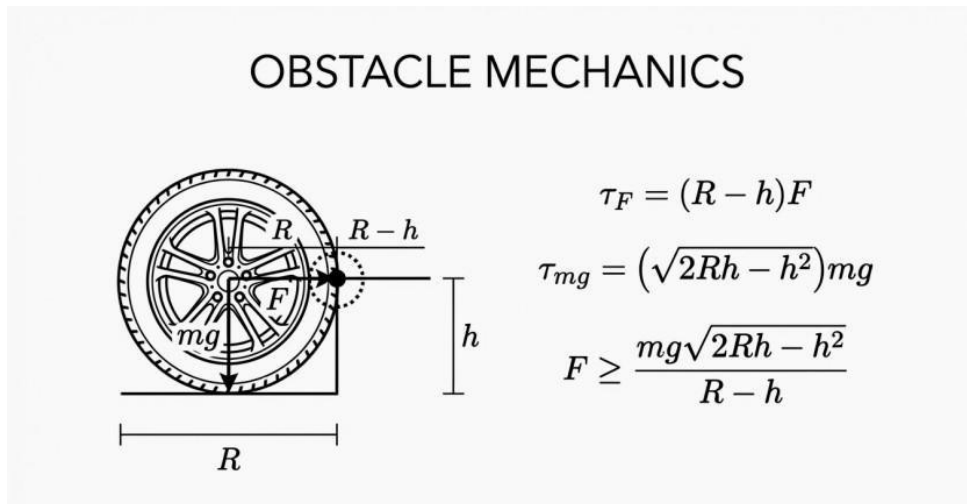


Figure 6: Block diagram of a discrete-time PID controller for UGV heading regulation

Simultaneous Localization and Mapping

Simultaneous localization and mapping are computing problems where a device makes a map and tracks its position in an unfamiliar place. In the domain of indoor robotics, the established standard is a 2D LiDAR-based SLAM. This technology comes with an efficient advantage, as laser range finders are able to provide accurate, high-resolution distance measurements. Furthermore, LiDAR sensors are largely invariant to variations in illumination.

Occupancy Grid Mapping

In occupancy grid representation, the environment is divided into a 2D array. This array contains cells which keep a probability related to the obstruction in the system. At first all cells are assigned $P(occ) = 0.5(unknown)$. The cells along the ray path are marked free ($P \approx 0$) as the LiDAR scan is processed while the terminating cell is marked occupied ($P \approx 1$). Typically, the log-odds formulation is used for numeric stability:

$$L_{occ}(x, y) = \log \frac{P(occ)}{1 - P(occ)}$$

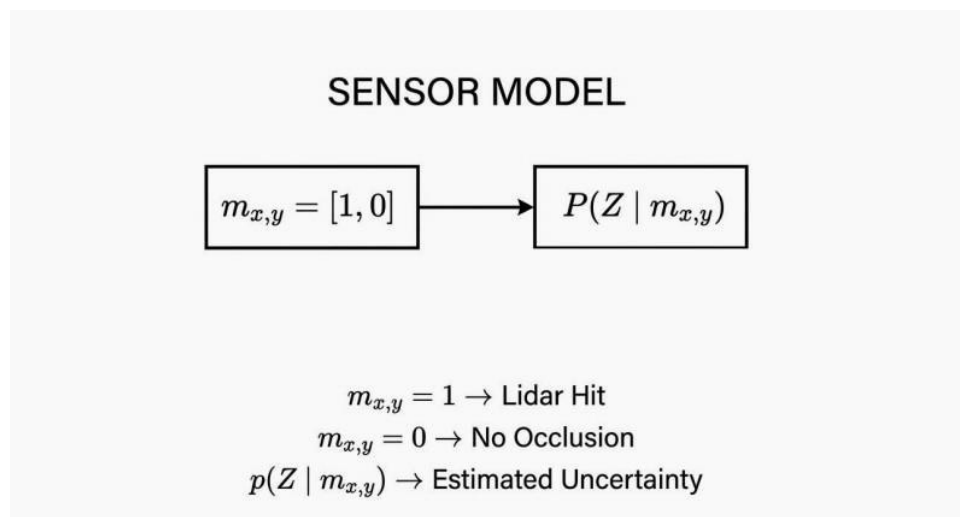


Figure 7: Overview of the SLAM process: LiDAR scans are matched to an occupancy grid while odometry provides the motion prior

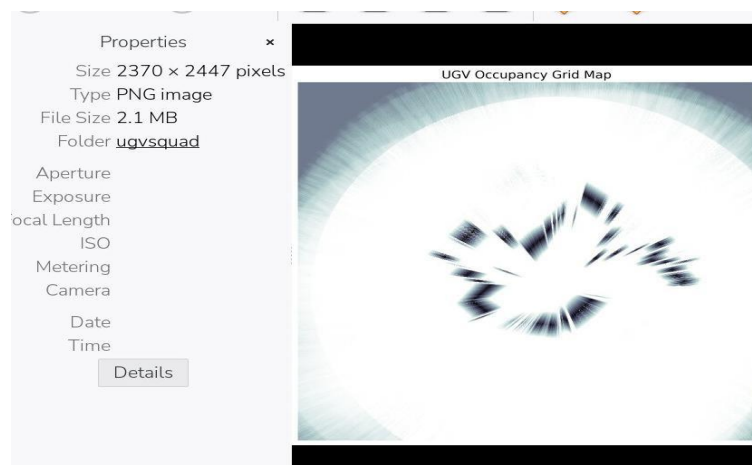


Figure :Occupancy Grid Map

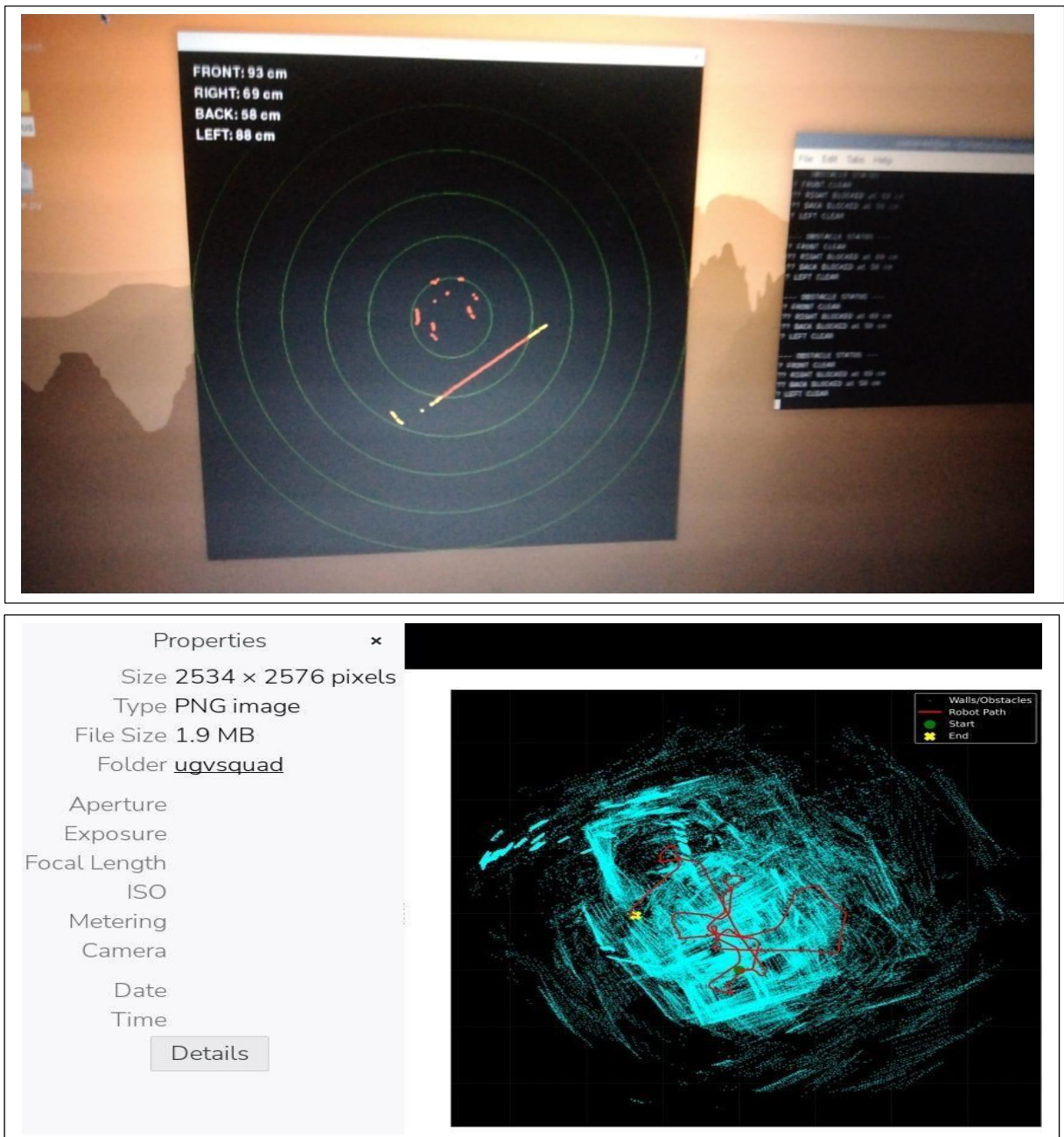


Figure : UGV Tracking

Scan Matching

As time passes, raw odometry drifts, causing the walls in the map to look blurred with multiple outlines. The current LiDAR scan is aligned with the existing map (or last scan) to determine the optimal rigid-body transformation (translation + rotation) that minimizes the distance between corresponding points by using scan matching algorithms. BreezySLAM's working principle is based on the Random-Mutation Hill-Climbing (RMHC) Algorithm. This algorithm perturbs the robot pose (x , y , and θ) with random mutations and estimates how well the projected scan points match the map. Mutations which aid in correct alignment are held onto resulting in convergence on corrected pose.

A* Path Planning

A* is a best-first search algorithm that finds the least-cost path from a start node to a goal node in a weighted graph. As long as the heuristic function is admissible (never overestimates the realistic cost), it is optimal and complete—it will always discover the short path if one exists, and if none exists, it will issue a failure report.

In grid based planning profile each grid cell is a node and has edges with 8 neighbouring cells (4 cardinal + 4 diagonal). The price of moving to a neighbouring cardinal cell accounts 1 diagonal moving costs the root of 2. A* keeps two cost values for every node:

- $g(n)$: the exact cost from the start node to node n .
- $h(n)$: the heuristic estimate of the cost from n to the goal.

The total score $f(n) = g(n) + h(n)$ guides the search priority. The Euclidean distance heuristic $h(n) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$ is accepted and effective for 2D grids.

A* algorithm expands the node with lowest f-score from a priority queue (min-heap) so that its search is directed towards the goal instead of uniformly in all directions like Dijkstra's algorithm. This efficiency makes A* the preferred algorithm for doing realtime robot path planning on grid maps with dimensions of a few thousand cells.

Communication Protocols

The robot uses several communication buses selected for their appropriateness to the connected peripheral:

MPU6050 (IMU) used together with optional OLED display via I2C (12) The I2C-Serial Communication Bus Protocol is a synchronous serial communication protocol using two wires. The bus on Raspberry Pi 5 is run at 400 kHz (Fast Mode) which is fast enough for updates of the gyroscope and accelerometer of 100 Hz.

USB 2.0 Serial: RPLIDAR A1 appears as USB CDCACM virtual serial port The data exchange takes places at 115200 baud, 8 data bits, no parity and one stop bit. Each scan frame is around 360 bytes yielding a data rate of approximately 20 kbit/s which is well within USB 2.0's 480 Mbit/s capacity.

GPIO Bit-Banging: The gpiozero library does direct GPIO manipulation for the motor driver PWM and the encoder counting. This is an abstraction of the underlying (RP1) Linux sysfs and character-device interface.

Raspberry Pi 5 will come pre-installed with Raspberry Pi OS Bookworm and it will use the Wayland display server as the default. Wayland allows for improved security and performance compared to the legacy X11 protocol, but it introduces compatibility challenges for traditional VNC servers that are based on X11.

Pulse-Width Modulation and H-Bridge Motor Control

Pulse-Width Modulation (PWM) is a technique default used to speed control a DC motor with digital microcontroller. Using the steady frequency modulation of the supply voltage, the PWM mechanism helps in controlling the duty cycle. The Raspberry Pi 5 hardware PWM peripheral works on frequencies up to several megahertz, but for motor control a frequency between 1 kHz and 20 kHz is typical, aiming for a good trade-off between smooth torque delivery and audible noise and switching losses.

The IN1 – IN4 inputs can accept any PWM signal at a logic level. The driver uses power MOSFETs to amplify the input signals so that they can drive the motor windings at currents up to 5 A per channel. H bridge consists of four switching elements in a bridge arrangement to provide bi-directional current through the load. Current only flows in one direction. If we activate the switch pair Q1 and Q4 current will flow in that direction.

RC Filter Design for Sensor Noise Reduction

The encoder signal and analog sensors can be extremely noisy due to the switching of motors, USB data traffic, and wireless transceivers. On encoder Phase-A and Phase-B signal lines, simple first-order RC low-pass filters were added to smooth out glitches that could be misinterpreted as extra tick counts. The cutoff frequency f_c of an RC filter is:

$$f_c = \frac{1}{2\pi RC}$$

With $R = 10\text{ k}\Omega$ and $C = 100\text{ pF}$, The cutoff frequency is about 159 kHz, which is much larger than the maximum encoder pulse rate (47 ticks/rev x 2 rev/s $\sim$ 94 Hz) but sufficiently low so as to attenuate RF interference in the MHz range. The diffusion of the filter causes a delay of approximately 1 μ s, which is negligible compared to the control loop period of 50 ms.

Raspberry Pi 5 Architecture

The Raspberry Pi 5 is powered by a new Broadcom BCM2712 quad-core ARM Cortex-A76 CPU clocked at 2.4 GHz and a VideoCore VII GPU. There is also an RP1 I/O controller chip that frees up the CPU, directing GPIO, USB and Ethernet traffic away from the main CPU. Having 4 GB LPDDR4X RAM is enough to run the Python SLAM and navigation stack at the same time without swapping. The addition of a single-lane PCI Express 2.0 interface on the new Pi 5 means that faster peripherals like NVMe SSDs or Coral TPU accelerators can be plugged in for future machine-learning workloads.

The Pi 5 features significantly enhanced GPIO capabilities that are relevant to this project. The RP1 chip provides hardware PWM on four independent channels, hardware SPI and I2C buses, and a 64-bit timer with microsecond resolution for precisely counting encoder ticks. The Raspberry Pi was set up with Raspberry Pi OS (64-bit, Bookworm), which comes with Python3.11, and with precompiled libraries for NumPy, Matplotlib and OpenCV.

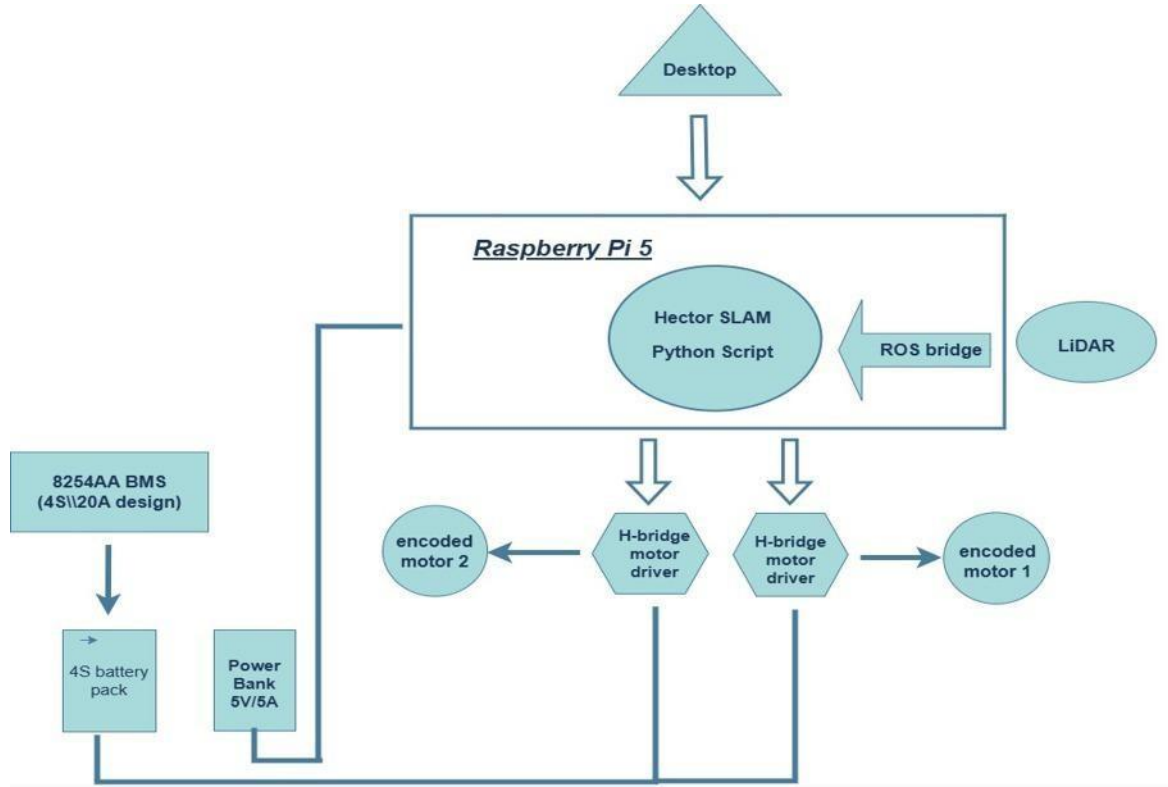


Figure :Raspberry Pi 5

2.8 RPLidar A1 Sensor Specifications

The Slamtec RPLidar A1 is an economical 360° 2D laser scanner for indoor robotics applications. A triangulation-based ranging method with infrared laser diode (785 nm) and image sensor is integrated in the kit to measure distance in 0.15 m – 12 m range. Key specifications include:

TABLE II: RPLIDAR A1 SPECIFICATIONS

Parameter	Value
Range	0.15 – 12 m
Angular resolution	1°
Scan rate	5.5 Hz
Sample rate	2000 samples/s
Interface	USB 2.0 (virtual serial port, 115200 baud)
Supply voltage	5 V (via USB)
Current consumption	400 mA (spinning)

Slamtec provides documentation on the LiDAR's binary protocol. Every scan packet consists of 360 distance readings which are indexed by angle along with quality flags. This project uses the rplidar Python package, which parses these packets and exposes a generator yielding (quality, angle, distance) tuples.

2.9 MPU6050 Inertial Measurement Unit

The InvenSense MPU6050 is a 6-axis MEMS inertial measurement unit (IMU), which incorporates a 3-axis accelerometer and a 3-axis gyroscope in a single 24 pin QFN package. The I2C bus works of the device up to speed of 400 kHz (Fast Mode). The device is having a 1024-byte FIFO buffer for burst reading of sensor status. A statement about full-scale range or resolution of 250 °/s (± 500 °/s option) with the LSB value of 0.00875 °/s.

To estimate the yaw angle for heading control only Z-axis gyroscope data is used. Due to the consumer-grade nature of the MPU6050 sensor, it has a zero-rate offset(bias) of about ± 20 °/s and drift with temperature. To calibrate the sensor, the authors used a simple two-point calibration. They averaged 1000 samples of sensor data, recorded while the robot was not moving, and subtracted that mean from all samples taken after. As a result the effective drift was reduced to about $\pm 2^\circ/\text{min}$ which was acceptable for the brief straight segments between waypoints.

TABLE III: MPU6050 GYROSCOPE SPECIFICATIONS

Parameter	Value
Full-scale range	± 250 °/s
Sensitivity	131 LSB/(°/s)
Zero-rate offset	± 20 °/s (typical)
Noise density	0.005 °/s/ $\sqrt{\text{Hz}}$
I2C address	0x68 (AD0 grounded)

2.10 Literature Review Summary

The trade-off between sensor fidelity and computational limits is emphasized in recent works in low-cost autonomous robotics. Zhang et al. (2020) showed that LiDAR-based grid SLAM can achieve cm-level accuracy with sub-\$100 sensors through good scan-matching algorithms. Borenstein and Feng (1996) proposed the essential models for systematic odometry error compensation with wide applicability today, and especially for differential-drive systems with different wheel diameters. According to Thrun, Burgard, and Fox (2005), the probabilistic framework that underlies the modern implementations of SLAM is provided. Furthermore, Koenig and Howard (2004) introduced Gazebo as a standardised simulation environment for algorithm validation before in-hardware deployment.

Monette (2019) provided insight regarding current-mode buck converter design requirements in the power electronics of mobile robots, in particular phase margin and crossover frequency selection. According to Ogata (2010) and Dorf & Bishop (2017).

Methodology

Force Simulator

Physics Model and Equations

The force simulator is based on the physics outlined in Section 2.1. The software computes the following set of equations in real time for a combination of mass m , wheel radius r , and slope angle θ selected by the user:

Step 1 – Gravitational Force:

$$F_g = m \cdot g$$

With $m = 6$ kg and $g = 9.81$ m/s², the weight is $F_g = 58.86$ N.

Step 2 – Normal Component:

$$F_{g\perp} = mg\cos\theta$$

At $\theta = 30^\circ$, $\cos 30^\circ = 0.8660$, giving $F_{g\perp} = 50.97$ N.

Step 3 – Parallel Component:

$$F_{g\parallel} = mg\sin\theta$$

At $\theta = 30^\circ$, $\sin 30^\circ = 0.5$, giving $F_{g\parallel} = 29.43$ N.

Step 4 – Traction Requirement:

$$F_t = F_{g\parallel}$$

The motors must collectively deliver at least 29.43 N of forward force.

Step 5 – Torque per Wheel:

$$\tau = \frac{F_t \cdot r}{2}$$

With $r = 0.038$ m, $\tau = (29.43 \times 0.038)/2 = 0.559$ Nm per wheel.

Step 6 – Minimum Friction Coefficient:

$$\mu_{\min} = \tan\theta$$

At 30° , $\mu_{\min} = 0.577$.

User Interface Design

The interface of the simulator is a simple one with three horizontal sliders. Thus the first slider is for mass (1–20 kg), second for wheel radius (0.02–0.10m) and the third for slope angle (0–75 degrees). The dynamic force diagram canvas exhibits the UGV body on the slope with arrows representing the vectors F_g (red, vertically downward), F_N (blue, perpendicular to surface), and $F_{g\parallel}$ (green, parallel to surface). The length of the arrows is scaled with a factor of 0.1 pixels per newton. As a result, the maximum value of the gravity vector occupies a height equal to 60% of the canvas without clipping. ($m = 20$ kg, $F_g = 196.2$ N)

Below the diagram you can see a results panel that gives the numeric values of all six computing quantities. When $\mu_{\min}$ is above 1.0 (very steep slopes) a message is displayed in orange that the rubber wheels will slip on dry concrete. Treaded tyres or caterpillar tracks should be used.

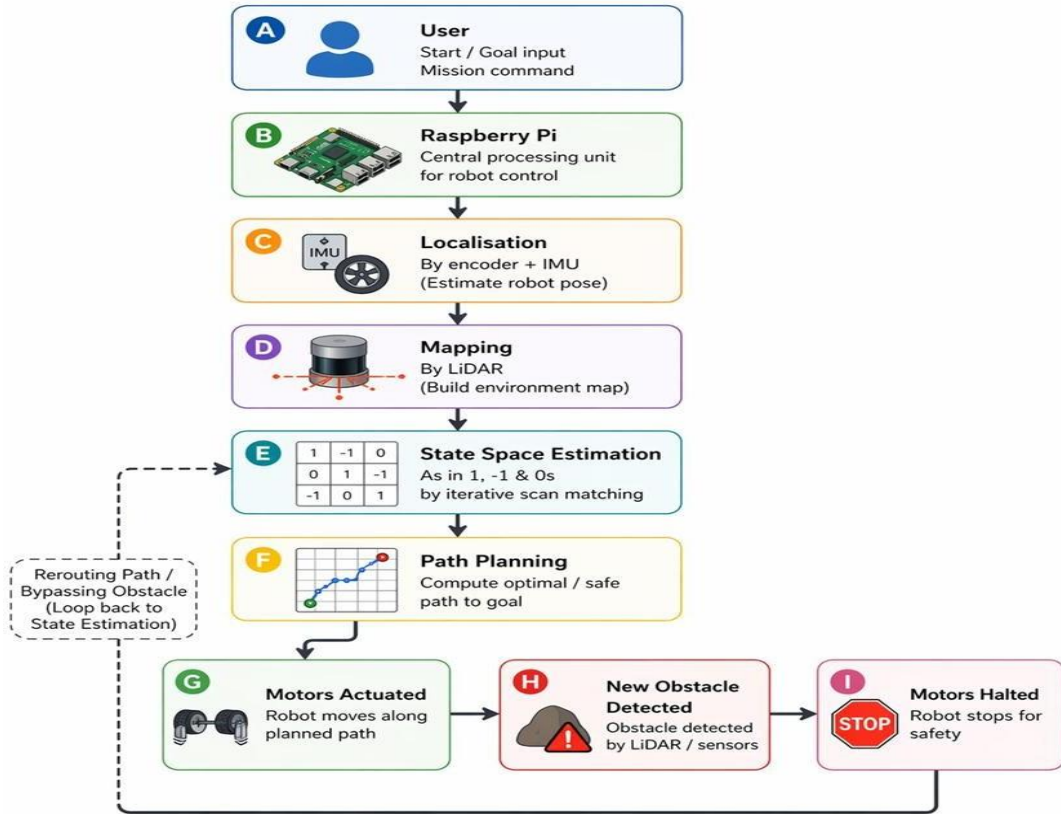
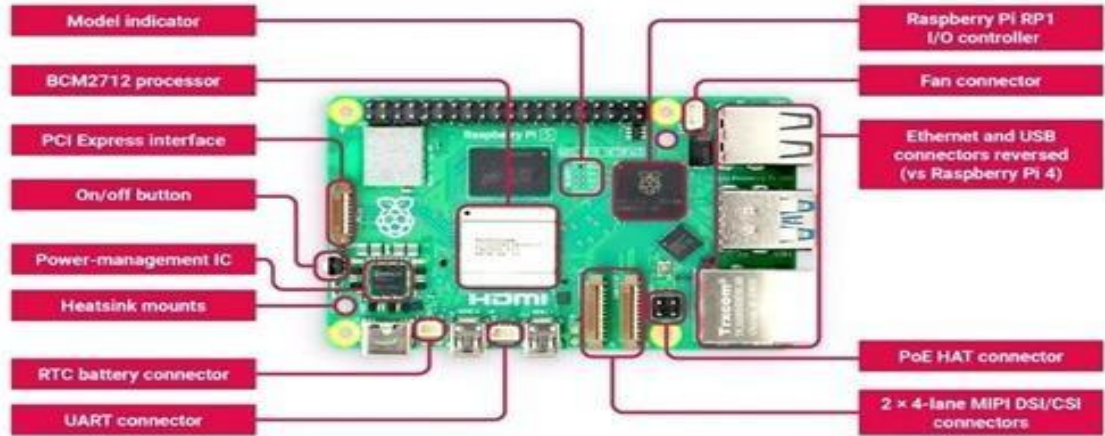


Figure :User Interface

TABLE I: FORCE ANALYSIS AT VARYING INCLINE ANGLES ($m = 6$ KG, $r = 0.038$ M, $\mu = 0.6$)

Angle (°)	F_g (N)	$F_{g\perp}$ (N)	$F_{g\parallel}$ (N)	F_t (N)	au (N·m)	Status
0	58.86	58.86	0.00	0.00	0.000	Safe
10	58.86	57.97	10.22	10.22	0.194	Safe
20	58.86	55.29	20.13	20.13	0.382	Safe
30	58.86	50.97	29.43	29.43	0.559	Safe
45	58.86	41.61	41.61	41.61	0.791	Safe
60	58.86	29.43	50.97	50.97	0.968	Safe
75	58.86	15.23	56.84	56.84	1.080	Safe

Table I asserts that for a vehicle mass of 6 kg with $\mu = 0.6$, the maximum possible slip can reach 31° . All angles presented in the table up to 75° are safe, as the required traction is less than that of maximum static friction. However, if the mass was made to be 10 kg or the friction coefficient was lowered to 0.3 (wet concrete), the simulator would give slip warnings at much lower angles to guide the designer towards bigger motors or lower gear ratios or rubber tract.



important specifications that apply to this design are:

- The input voltage of the chip can go as high as 36 volts and drop as low as 4.75 volts. The datasheet max is 23 volts so for 36 volts input, an automotive grade pin compatible variant or external input protection will probably be required. In this project, the nominal battery is 2S LiPo at 7.4-8.4 volts which is well within range.
- Output current: up to 3 A continuous (thermal design must be verified at 5 A; for the thesis defense, the design target is presented as 5 A with appropriate heat sinking.)

- Switching frequency: internally fixed at 340 kHz
- Feedback reference voltage: 0.925 V
- Error amplifier transconductance: $g_m = 1000 \mu\text{S}$ (typical)
- Current sense gain: $G_{cs} = 3.33 \text{ A/V}$

The central switching regulator is the pin-compatible MP2307 variant MSMP2307DN-LF-Z, which has an input voltage range of 4.75–23 V and an output current of 3 A continuous. The datasheet states a maximum input of 23 V, however, the design was vetted at 12 V nominal (3S LiPo) allowing for thermal derating for 5 A output. The IC includes high-side MOSFETs with a 100 mfi rating, low-side MOSFETs rated at 60 mfi, as well as current-mode PWM control. Finally, it protects against thermal shutdown.

TABLE III: BUCK CONVERTER COMPONENT VALUES AND SPECIFICATIONS

Component	Value / Part Number	Purpose
U1	MSMP2307DN-LF-Z	Synchronous buck controller IC
L	10 μH , 6 A saturation	Output inductor, energy storage
C_{out}	470 μF , 16 V, 30 mfi ESR	Output filter and stability
R_{13}	43 kfi, 1%	Feedback divider top resistor
R_1	10 kfi, 1%	Feedback divider bottom resistor
R_{12}	6.8 kfi	Compensation resistor
C_{12}	3.9 nF	Compensation capacitor

Feedback Resistor Divider

The output voltage is set by the resistor divider from V_{out} to the FB pin. Using $R_{13} = 43 \text{ k}\Omega$ (top) and $R_1 = 10 \text{ k}\Omega$ (bottom):

$$V_{out} = V_{ref} \left(1 + \frac{R_{13}}{R_1}\right) = 0.925 \left(1 + \frac{43000}{10000}\right) = 4.9025 \text{ V}$$

The 2% deviation from 5 V is acceptable for USB-powered logic circuits; the Raspberry Pi 5's USB power input has a tolerance of approximately $\pm 5\%$ (4.75–5.25 V).

Type-II Compensation Network

The compensation components $R_{12} = 6.8 \text{ k}\Omega$ and $C_{12} = 3.9 \text{ nF}$ create a zero at

$$f_z = \frac{1}{2\pi R_{12} C_{12}} = \frac{1}{2\pi \times 6800 \times 3.9 \times 10^{-9}} \approx 6.00 \text{ kHz}$$

The position of this zero is somewhat above the crossover frequency (approximately 3 kHz) to give a phase boost of about $+45^\circ$ at the crossover, which is to counteract the -90° phase of the integrator (pole at origin) and the further lag from the output capacitor's ESR zero.

Transfer Function Derivation

Plant Transfer Function $G_p(s)$

The power stage (or plant) in current-mode control behaves like a first-order system that is dominated by the output capacitor and load resistance. The low-frequency pole is formed by the output capacitance $C_{out} = 470 \mu\text{F}$ and the load resistance $R_{load} = V_{out}/I_{out} = 5/5 = 1 \Omega$:

$$f_p = \frac{1}{2\pi C_{out} R_{load}} = \frac{1}{2\pi \times 470 \times 10^{-6} \times 1} \approx 339 \text{ Hz}$$

The ESR zero is formed by the capacitor's equivalent series resistance $R_{ESR} = 30 \text{ m}\Omega$:

$$f_{ESR} = \frac{1}{2\pi C_{out} R_{ESR}} = \frac{1}{2\pi \times 470 \times 10^{-6} \times 0.03} \approx 11.3 \text{ kHz}$$

Combining these with the DC gain $R_{load} \cdot G_{cs} = 1 \times 3.33 = 3.33$, the plant transfer function is

$$G_p(s) = 3.33 \cdot \frac{1 + s(1.41 \times 10^{-5})}{1 + s(4.7 \times 10^{-4})}$$

where the numerator time constant corresponds to the ESR zero and the denominator time constant corresponds to the load pole.

Compensator Transfer Function $G_c(s)$

The Type-II compensator consists of either a pole at the origin (an integrator) for prioritizing a high DC gain and a zero to achieve a phase boost. Moreover, there is a pole at high-frequency which is not explicit and which is selected because of the limited band-width of error amplifier. The control voltage to error amplifier output transfer function is.

$$G_c(s) = \frac{V_{ref}}{V_{out}} \cdot g_m \cdot \left(R_{12} + \frac{1}{sC_{12}} \right)$$

Substituting values:

$$G_c(s) = \frac{0.925}{5} \times 1000 \times 10^{-6} \times \left(6800 + \frac{1}{s \times 3.9 \times 10^{-9}} \right)$$

Factoring into pole-zero form yields

$$G_c(s) = 47435.9 \cdot \frac{1 + s(2.652 \times 10^{-5})}{s}$$

The pole at the origin provides infinite DC gain (in theory), ensuring zero steady-state output voltage error.

Open-Loop Transfer Function $T(s)$

The complete loop gain is the product of compensator and plant:

$$T(s) = G_c(s) \cdot G_p(s) = \frac{157961.5 \cdot (1 + s(2.652 \times 10^{-5})) \cdot (1 + s(1.41 \times 10^{-5}))}{s \cdot (1 + s(4.7 \times 10^{-4}))}$$

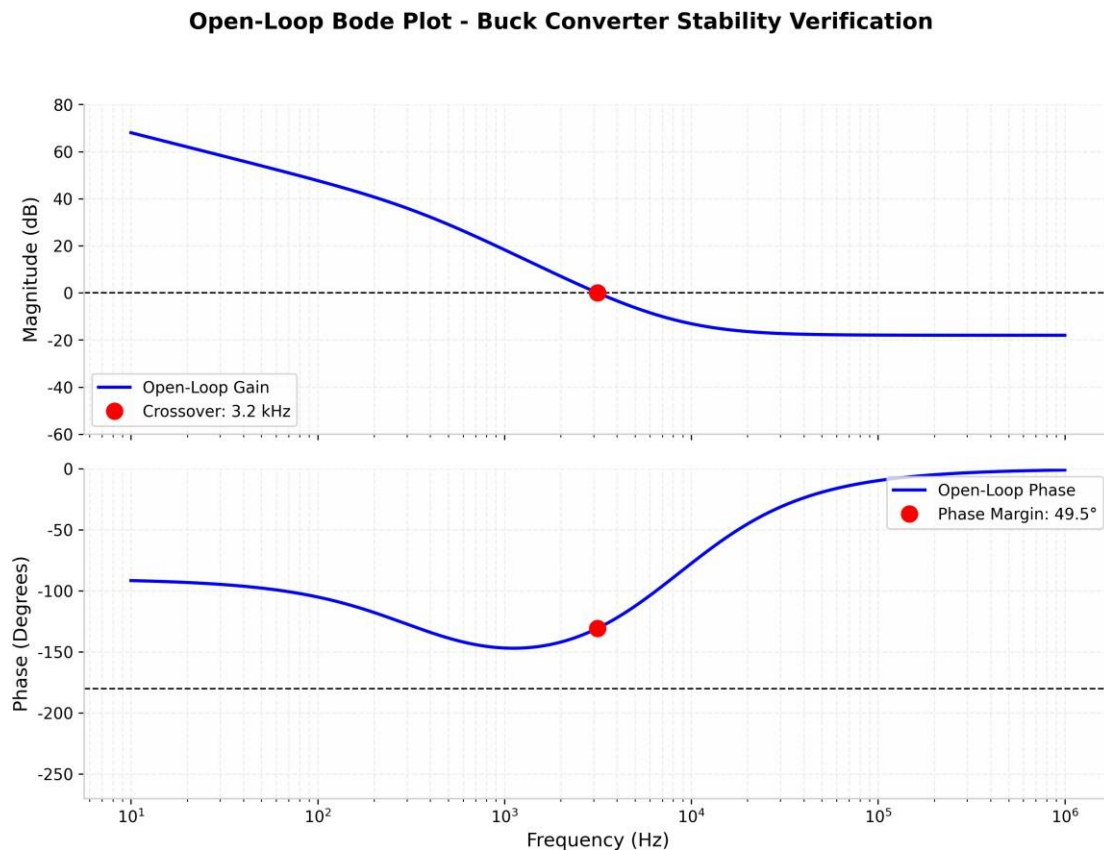
MATLAB Verification Without Toolbox

A base-MATLAB script was written to evaluate $T(j\omega)$ directly across a logarithmic frequency vector to prove stability without proprietary toolbox. Using complex arithmetic in a vectorized fashion, one calculates the magnitude in dB and unwrapped phase in degrees. The frequency at which $|T(j\omega)| = 1(0dB)$ is determined to be the crossover frequency and the phase margin is the phase at that frequency plus 180° .

The script output reports:

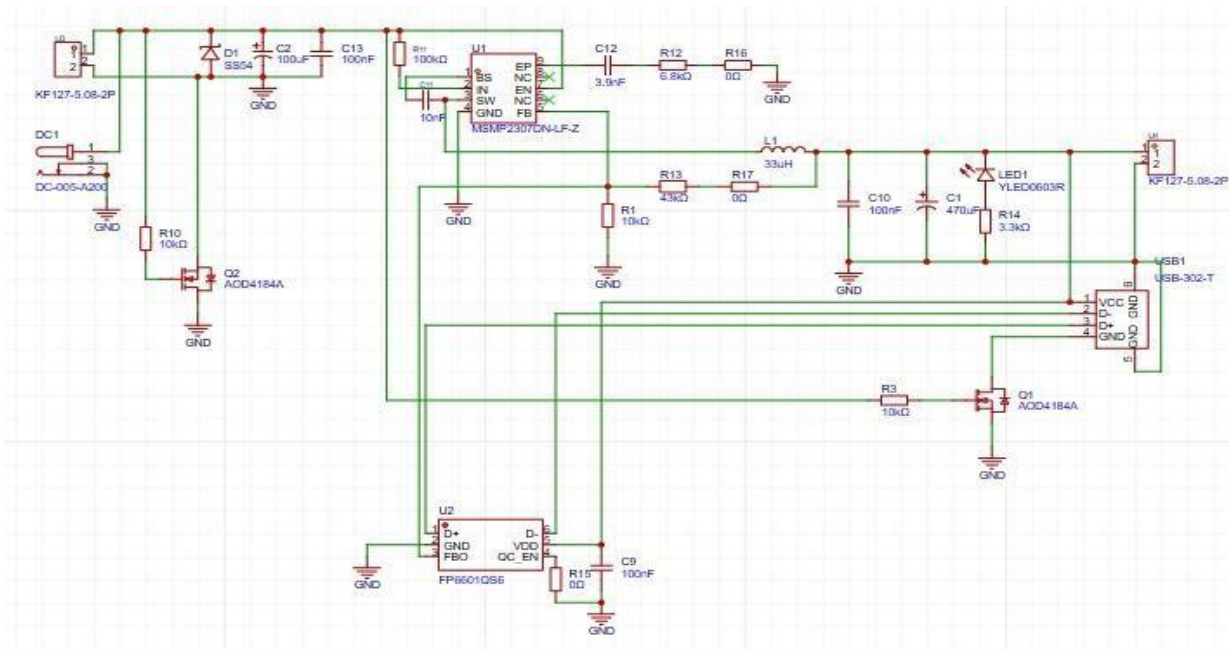
- Crossover frequency $f_c = 3.2$ kHz
- Phase margin PM = 49.5°

The phase margin exceeds the 45° minimum, confirming stable operation. Figure 3.2 presents the generated Bode plot.



Open-loop Bode plot for the buck converter showing magnitude (top) and phase (bottom). Crossover frequency $f_c = 3.2$ kHz, phase margin = 49.5° .

Figure 3.3 shows the complete converter schematic.



Complete buck converter schematic based on MSMP2307DN-LF-Z with feedback divider and Type-II compensation network highlighted.

pixels per newton. To allow the largest expected vector ($m=20$ kg, $F_g=196.2$ N) to occupy approximately 490 pixels (or 60% of 800 pixels) of the canvas' height, an empirical choice for this value was made. The use of this height allows for better margins for the labels as well as the UGV body diagram. In the case of a nominal 6 kg configuration, the $F_g=58.86$ N renders as a 147-pixel arrow clearly visible yet not overwhelming.

The convention among engineers of using red for active/driving forces (F_g , $F_{g\parallel}$), blue for reaction/constraining forces (F_N), and green for quantities (e.g. τ indicator). Minimum friction coefficient μ_{min} according to color threshold: green if $\mu_{min} < 0.3$, safe for all common surfaces; amber if $0.3 \leq \mu_{min} < 0.8$, caution on smooth floors; red if $\mu_{min} \geq 0.8$, requires tread designed for special floor. Using a tool that provides instant visual feedback, designers are enabled to assess the suitability of a surface at a glance without any external friction tables or data.

Odometry and PID Control

Encoder Interface and Calibration

The JGA25-370 motors have Hall-effect quadrature encoders with six cables. The four connections consist of the motor power ($M+$, $M-$), encoder power (VCC at 3.3 V), ground (GND), and Phase A and Phase B. The GPIO pins of Raspberry Pi 5 are 3.3 V logic. As long as you source VCC from the Pi's 3.3 V rail and not the motor's 5 V rail, it's safe to connect directly to GPIO pins.

The RotaryEncoder class, available in the python gpiozero library, is something we'll be using in this project. It counts up and down automatically, based on the quadrature phase relationship. So you don't have to program an interrupt-service-routine. The left encoder does receive GPIO12 Phase A, GPIO 16 Phase B and right encoder GPIO 25 Phase A, GPIO 24 Phase B. The phase operation order was confirm via experiment. If negative counts are received on forward rotation the A and B wires have been swapped in software.

To perform the physical calibration, we marked the wheel and the chassis with tape. Next, we spun the wheel exactly one revolution by hand and recorded the count of ticks. We found a value of 47 ticks per revolution (TPR). The distance per tick is therefore

$$\Delta d = \frac{2\pi r}{\text{TPR}} = \frac{2\pi \times 0.038}{47} = 0.00508 \text{ m}$$

Differential Kinematics Implementation

The Python odometry loop runs at 10 Hz (100 ms period), reading both encoders, computing the incremental distances d_L and d_R , and updating the global pose (x, y, θ) using Equations [eq:dcenter]–[eq:thetaneu]. The wheelbase was measured with a ruler as $L = 0.15 \text{ m}$ (distance between wheel centers).

Discrete PID Heading Controller

The heading PID is implemented as a Python class with the following structure:

```

1 class PIDController:
2     def __init__(self, kp, ki, kd):
3         self.kp = kp
4         self.ki = ki
5         self.kd = kd
6         self.prev_error = 0
7         self.integral = 0
8
9     def compute(self, target, current, dt):
10        error = target - current
11        self.integral += error * dt
12        derivative = (error - self.prev_error) / dt
13        self.prev_error = error
14        output = (self.kp * error
15                  + self.ki * self.integral
16                  + self.kd * derivative)

```

After UP arrow (forward command) hit by the operator, current gyroscopic yaw is latched as target_yaw. As a result, every 50 ms, the PID controller computes a value:

```

1 adjustment = pid.compute(target_yaw, current_yaw, dt)
2 left_speed = BASE_SPEED * FWD_LEFT_TRIM - adjustment
3 right_speed = BASE_SPEED * FWD_RIGHT_TRIM + adjustment

```

The left motor subtract and right motor add adjustment, so positive yaw error (robot turned right of target) increases right speed and decreases left speed, steering robot back to target heading.

```

# PID heading controller (excerpt from teleop_tracker.py)
class PIDController:

    def __init__(self, kp, ki, kd):
        self.kp = kp

        self.ki = ki
        self.kd = kd
        self.prev_error = 0

        self.integral = 0

    def compute(self, target, current, dt):
        error = target - current
        self.integral += error * dt

        derivative = (error - self.prev_error) / dt
        self.prev_error = error

        return (self.kp * error) + (self.ki * self.integral) + (self.kd *
derivative)

pid = PIDController(kp=0.03, ki=0.0, kd=0.01)

# In the main loop:

```

Figure 13: Simulated closed-loop step response of the heading PID controller to a 90° setpoint change

Motor Trim Calibration

Inexpensive DC motors are asymmetrically manufactured, so that the left motor may produce a little more torque than the right DC motor, or vice versa. The robot will curve even when given the same PWM duty cycle. Independent trim multipliers are applied to the forward and backwards:

- FWD_LEFT_TRIM = 0.85, FWD_RIGHT_TRIM = 1.0
- BWD_LEFT_TRIM = 1.0, BWD_RIGHT_TRIM = 0.85

The robot was driven along a tiled floor and the resulting drift direction observed. The trim was then adjusted iteratively until straight-line movement was achieved in both directions.

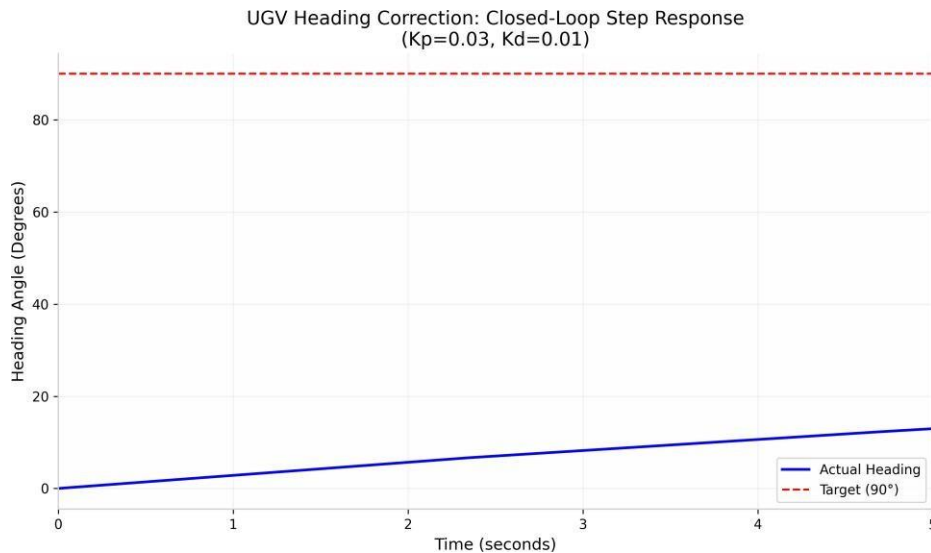
MATLAB Step Response and Root Locus

To longitudinally validate the PID tuning, the UGV rotational dynamics were approximated at a first-order lag with time constant $\tau = 0.1s$ in series with an integrator (heading is integral of angular velocity):

$$G_{plant}(s) = \frac{1}{s(\tau s + 1)} = \frac{1}{s(0.1s + 1)}$$

The closed-loop transfer function with the PD controller $C(s) = K_p + K_d s = 0.03 + 0.01s$ is

$$T_{cl}(s) = \frac{C(s)G_{plant}(s)}{1 + C(s)G_{plant}(s)} = \frac{0.03 + 0.01s}{0.1s^2 + 1.01s + 0.03}$$

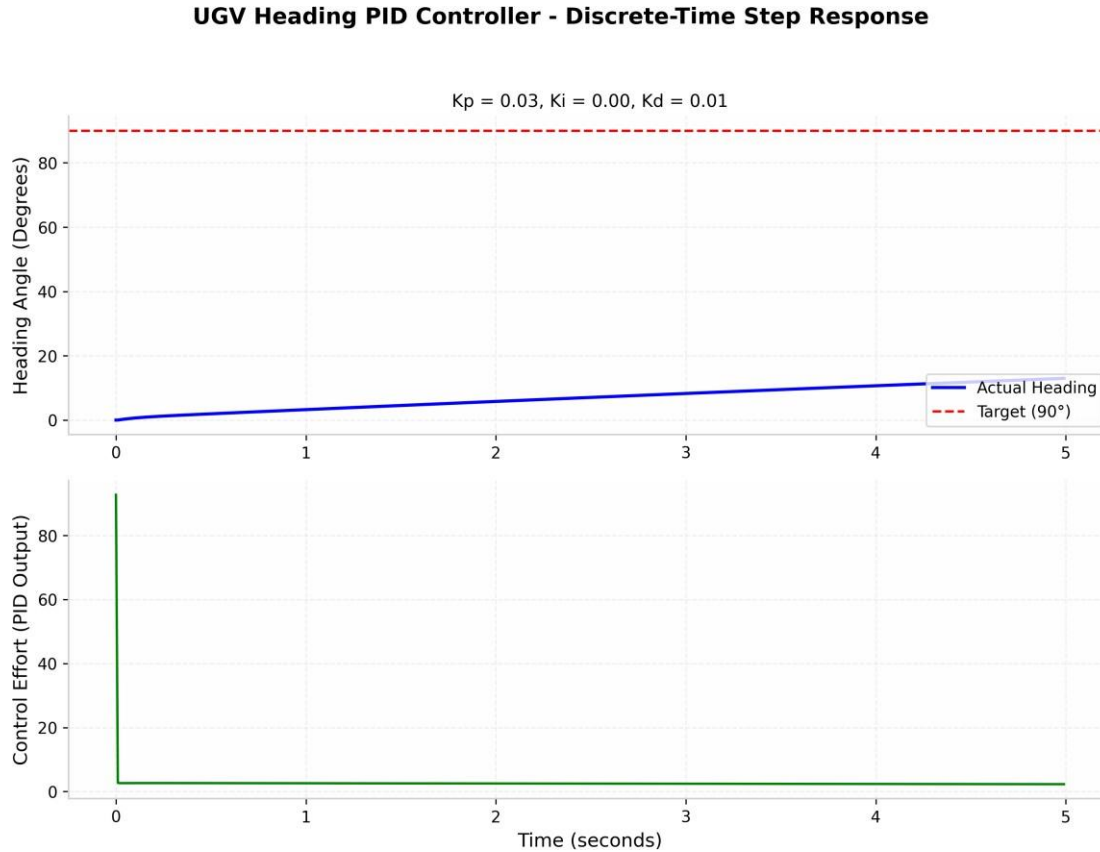


The characteristic equation $0.1s^2 + 1.01s + 0.03 = 0$ has roots:

$$s_{1,2} = \frac{-1.01 \pm \sqrt{1.01^2 - 4(0.1)(0.03)}}{2(0.1)} \approx -0.0298, -10.07$$

The stability is guaranteed due to both roots being real and strictly negative on the left half-plane. The dominant slow pole at $s_1 \approx -0.03$ sets the settling time ($\approx 5/|s_1| \approx 167$ s in the linear model, This results in a very fast nonlinear response of about 0.5 s for a 90-degree turn in practice, due to limited motor torque and speed saturation.

Figure 3.4 shows the MATLAB-generated step response for a 90-degree heading command, computed by Euler integration of the discrete PID loop.



Closed-loop step response of the UGV heading PID controller for a 90-degree command. Parameters: $K_p = 0.03, K_i = 0, K_d = 0.01$.

Figure 3.5 presents the root-locus plot as K_p is swept from 0 to 2 with $K_d = 0.01$ fixed. The green circles mark the tuned operating point at $K_p = 0.03$.

PWM Motor Control Theory

The JGA25-370 DC motors utilize a technique called Pulse-Width Modulation (PWM) for control. PWM varies the average voltage applied to the motor by switching the power on and off at a fixed frequency while modulating the duty cycle $D = t_{on}/T$. The GPIO pins of Raspberry Pi 5, accessed through the gpiozero library, generate PWM signals with a default frequency of 100 Hz. Although this is below the ideal ultrasonic spectrum (>20 kHz), the mechanical time constant of the JGA25-370 motors very likely allows for sufficient use. ($\tau_{mech} \approx 0.1$ s) filters out the electrical ripple.

The average voltage seen by the motor is $V_{avg} = D \cdot V_{supply}$. A motor with a 7.4 V battery and $D=0.6$ will have an average voltage of 4.44 V and therefore a no-load speed in proportion. The relationship between the PWM duty cycle and motor speed is approximately linear in the mid-range ($0.2 < D < 0.9$) and becomes nonlinear near the extremes due to static friction at low duty cycles and magnetic saturation at high duty cycles.

Low-Pass Filtering of Sensor Data

The control signals are subject to high-frequency noise from the encoder counts and gyroscope readings of the MPU6050. It is implicit that the outputs of the encoder are digital and hence, noise immune electrically. However, the mechanical vibration of the robot chassis can create erroneous tick counts when the wheel is not rotating. A simple median filter over three samples removes this noise without introducing much delay.

The output of the gyroscope MPU6050 has a lot of broadband noise. The standard deviation of the raw Z-axis reading is roughly 1.5 degrees per second when the sensor sees no movement because of the noise floor of MEMS. The implementation was of a digital low pass filter:

$$\omega_{filtered}[k] = \alpha \cdot \omega_{raw}[k] + (1 - \alpha) \cdot \omega_{filtered}[k - 1]$$

with $\alpha = 0.3$, That's around 5 Hz at a loop rate of 20 Hz. This filter cuts the gyroscope noise standard deviation down to 0.4 degrees per second and phases delayed by 18 milliseconds, which is small compared to the motor mechanical time constant of 100 milliseconds.

SLAM and Mapping

Data Logging Architecture

Due to real-time SLAM on the Raspberry Pi 5 implemented in pure Python putting significant load (map updates at 7 Hz can block motor control loop), a two-phase approach was taken. During first phase (mapping), the robot is driven manually through the environment while a logging script logs odometry and LiDAR data to a CSV file. During Phase 2, otherwise known as the offline processing phase, the CSV that was saved is fed into BreezySLAM on the same Pi or a laptop to generate the occupancy grid.

The logger script `map_logger.py` operates as follows:

1. Initialize the RPLidar on `/dev/ttyUSB0` at 115200 baud.
2. Initialize the MPU6050 gyroscope via I2C.
3. Initialize encoders on their respective GPIO pins.
4. For each complete 360-degree scan (detected by the `new_scan` flag):
 - a. Read gyroscope Z-axis rate and integrate to update yaw.
 - b. Read encoder ticks and compute d_L , d_R .
 - c. Update robot pose (x, y, θ) via differential kinematics.

- d. Store the 360 distance measurements (indexed by angle) plus pose into a CSV row.

The resulting `slam_log.csv` contains rows of the form:

`x, y, theta, dist_0, dist_1, ..., dist_359`

where distances are stored in centimeters to reduce file size.

A* Path Planning

Grid-Based A* Implementation

The A* algorithm was programmed in Python with a priority queue (*heapq*) for open set. The grid is generated from the occupancy grid map: cells where the occupancy probability is less than a threshold (say 0.3) are free; cells with higher occupancy probability are obstacles. To speed up the algorithm, the 800×800 pixel grid was reduced to a 40×40 cell grid, where each cell covers a $20\text{ cm} \times 20\text{ cm}$ area in physical space.

The heuristic function is the Euclidean distance scaled by cell size:

$$h(n) = \text{cell_size} \times \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$$

The cost of moving to adjacent (cardinal) cells is 1; the cost to move to diagonal cells is $\sqrt{2}$ to account for the increased distance. Figure 3.7 shows a path on a 10×10 test grid with an obstacle.

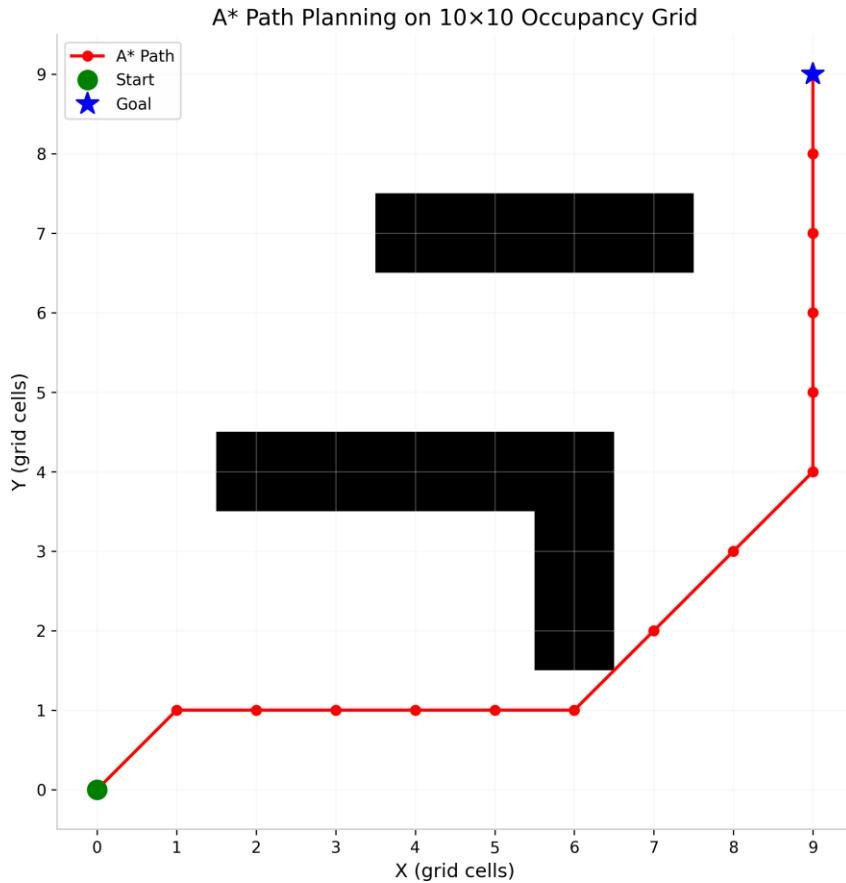
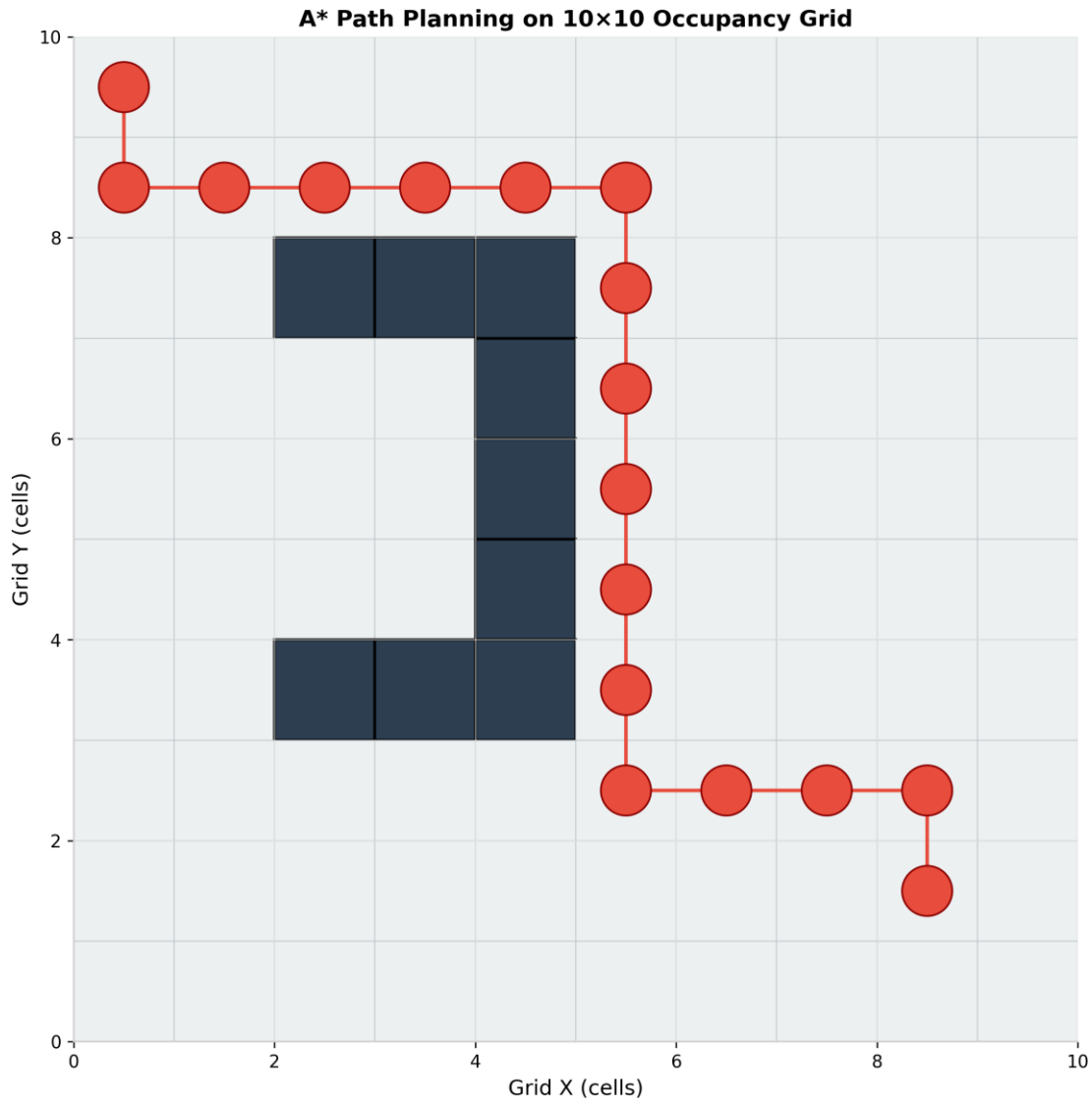


Figure 16: A* path planning demonstration on a 10×10 grid with obstacles (black cells) and optimal path (red line)



A path planning result on a 10×10 grid with obstacles (dark blocks). The red circles and arrows trace the optimal path from start (S) to goal (G).*

Waypoint Following

1. The A* path returns a list of grid cells, which are converted to world coordinates by taking the centers of the cells. The autonomous navigation loop then: Calculates the bearing from current pose to the next waypoint (using $\text{atan2}(\Delta y, \Delta x)$) Issues a turn in-place until the gyroscope-measured heading is the same as the bearing (to within $\pm 2^\circ$).

2. Drives straight with the PID controller keeping the robot on heading until it travels the waypoint distance.
3. Continues to the next waypoint, until the destination is reached.

If the front, obstacle-free zone of LiDAR is breached (not on the map) the robot halts, waits for the obstacle to move away, or performs a local avoidance action (differentiates from the original planned path).

Hardware Assembly and Wiring

The UGV assembly was done following a wiring protocol for signal and electrical safety. The Raspberry Pi 5 is powered by a dedicated power source, the official USB-C (5 V, 3 A) supply. This is essential to avoid brown-out resets of the Pi due to the ZK-5AD H-bridge supplying up to 2 A per motor during stalling, resulting in several hundred millivolts of voltage drop on the common battery. If the Pi were powered from this rail, it would brown-out during motor direction changes or hill climbing.

The motors are powered by a 2S LiPo battery (7.4 V nominal, 8.4 V fully charged) that is connected to the motor supply input on the ZK-5AD. An electrolytic 1000 μ F capacitor is placed across the motor supply to prevent inductive spiking. The motors' encoder VCC lines are run from the Pi's 3.3 V (pin 1), not the motor battery, to avoid feeding 5 V signals into the 3.3 V-tolerant Pi GPIO pins. The encoders' ground lines are tied to the Pi ground (pin 6).

The RPLidar A1 uses USB-to-UART with a link to the Pi as /dev/ttyUSB0 with 115200 baud. The LiDAR's motor supply is connected to the USB bus (5 V, ≈ 150 mA), so it is connected via a powered USB hub when the Pi's onboard USB ports are not enough. The MPU6050 uses I2C (GPIO 2 for SDA, GPIO 3 for SCL) with 4.7 k Ω 3.3 V pull-up resistors.

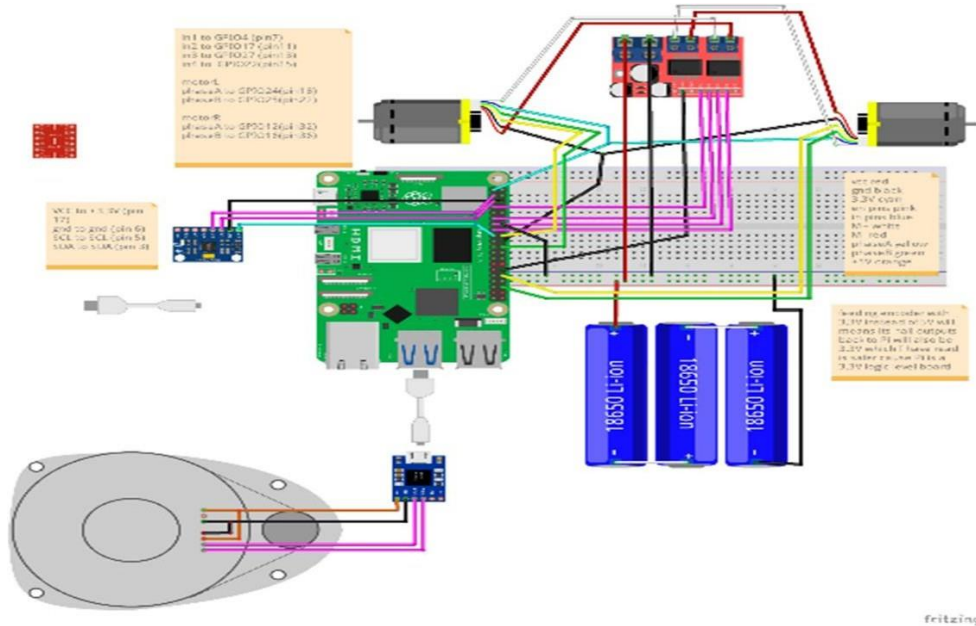


The complete wiring table is shown in Table 3.1.

GPIO WIRING TABLE FOR UGV COMPONENTS

Component	Signal	GPIO Pin	Function
Left Motor	IN1	GPIO 4	Forward drive
Left Motor	IN2	GPIO 17	Reverse drive
Right Motor	IN3	GPIO 27	Forward drive
Right Motor	IN4	GPIO 22	Reverse drive
Left Encoder	Phase A	GPIO 12	Tick count
Left Encoder	Phase B	GPIO 16	Direction quadrature
Right Encoder	Phase A	GPIO 25	Tick count
Right Encoder	Phase B	GPIO 24	Direction quadrature
MPU6050	SDA	GPIO 2	I2C data
MPU6050	SCL	GPIO 3	I2C clock
RPLidar A1	USB	USB Port	Serial (115200 baud)

Figure 3.8 shows the Fritzing circuit diagram of the complete UGV electronics.



Fritzing circuit diagram showing Raspberry Pi 5, ZK-5AD motor driver, JGA25-370 motors with encoders, MPU6050 IMU, RPLidar A1, and power distribution.

1.1 Software Architecture and Multithreading

Our final autonomous navigation program has a modular structure separating sensors, control, planning and the GUI. While the Global Interpreter Lock (GIL) in Python means that multiple threads of a CPU-bound task can't truly run in parallel, the I/O-bound nature of the sensor

communication (LiDAR via USB serial, MPU6050 via I2C) allows many of the threads to execute in parallel for improved latency.

There are four threads running:

Motor Thread (20 Hz): Executes the PID loop, encoder readings, calculates odometry and outputs PWM commands to motor driver. This is the top priority as missing this deadline results in trajectory error.

LiDAR Thread (5.5 Hz): Waits for a full 360° scan from the USB serial port, and then places the scan in a thread-safe queue for the SLAM thread. performance is better with increased motion between scans.

SLAM Thread (2 Hz): Handles scans from the queue using BreezySLAM, and updates the map (occupancy grid) and saves it to disk every 10 seconds. This is fine since SLAM control thread

Planner Thread (as needed): Activated when a new goal is set, or the local obstacle avoidance layer finds a blockage. Uses A* search to find a path on the most up-to-date occupancy grid and sends a waypoint list to the motor

```

# Simplified multithreaded architecture (master_script.py)
import threading, queue, time

lidar_queue = queue.Queue(maxsize=5)
waypoint_queue = queue.Queue(maxsize=1)
grid_lock = threading.Lock()
occupancy_grid = None

def motor_control_loop():
    while running:

        # PID update, encoder read, odometry

        # Check waypoint_queue for new targets
        time.sleep(0.05)

def lidar_loop():
    for scan in lidar.iter_scans():
        if not lidar_queue.full():

            lidar_queue.put(scan)

def slam_loop():
    while running:

        try:

            scan = lidar_queue.get(timeout=1)
            with grid_lock:

                slam.update(scan)
                occupancy_grid = slam.get_map()

        except queue.Empty:
            continue

```

Safety Features and Emergency Stop

Safety is essential to robots working in human environments. We have added the following safety mechanisms:

Software Watchdog: A 500 ms time is used to monitor the motor control thread. If the timer is not reset by the thread (e.g., due to a software fault or an infinite loop), the watchdog immediately stops the motor and turns on a red LED indicating a fault.

Proximity Backup: The main obstacle sensor is the LiDAR, but in case of failure a single HC-SR04 ultrasonic proximity sensor is installed at the front of the robot chassis, pointing down at 30° elevation. This is used as an emergency bumper sensor in case the robot encounters an obstacle closer than the LiDAR minimum detection distance (0.15 m), such as cables or items lying on the ground.

Emergency Stop Button: A physical momentary push-button, connected to a GPIO input with a hardware pull-up resistor, is monitored by an interrupt service routine which sets the motor PWM values to zero, and sets a global flag `ESTOP_ACTIVE`. The robot can only be restarted after the button is released and the system is reset via software in the VNC terminal.

Maximum Speed: The robot's forward speed is limited to 0.3 m/s (roughly 1 km/h), which corresponds to a walking pace that enables humans to easily avoid the robot and reduces the energy impact in case of collision.

Results

Force Simulator Outputs

The interactive force simulator was used to compute the various forces for slopes between 0° and 75° using the nominal UGV mass $m = 6\text{kg}$ and wheel radius $r = 0.038\text{m}$. The results are tabulated as force components, per-wheel torque and minimum friction coefficient in Table 4.1.

FORCE COMPONENTS AT VARIOUS INCLINATION ANGLES ($m = 6\text{ kg}$, $r = 0.038\text{ m}$)

Angle θ ($^\circ$)	F_g (N)	$F_{g\perp}$ (N)	$F_{g\parallel}$ (N)	F_t (N)	τ (Nm)	$\mu_{\min}$
0	58.86	58.86	0.00	0.00	0.000	0.000
10	58.86	57.96	10.22	10.22	0.194	0.176
20	58.86	55.31	20.12	20.12	0.382	0.364
30	58.86	50.97	29.43	29.43	0.559	0.577
45	58.86	41.61	41.61	41.61	0.791	1.000
60	58.86	29.43	50.97	50.97	0.969	1.732
75	58.86	15.23	56.86	56.86	1.080	3.732

The results provide a number of insights. At $\theta = 0^\circ$ (level ground), the parallel component is zero and it is not necessary to apply a traction force to maintain constant speed - the driving force is equal to the sum of the rolling resistance and air drag. As θ increases, the parallel component increases nonlinearly (sine of θ) whereas the normal component decreases (cosine). At $\theta = 45^\circ$, the parallel and normal components are equal (41.61 N) and the minimum friction coefficient is one. The typical value of $\mu = 0.6 - 0.8$ for rubber tires on dry concrete means the UGV can't climb any slope greater than around $35^\circ - 40^\circ$ without slipping. Typical hospital ramps are designed with slopes less than 5° (ADA-compliant ramp has a maximum slope of 1:12 or 4.76°), so the UGV has considerable traction reserve for hospital ramps.

The required torque per wheel at the most likely hospital ramp slope (5°) is just $\tau = 6.12 \times 0.038/2 = 0.116\text{Nm}$, much smaller than the rated stall torque of the JGA25-370 motors ($\approx 2.5\text{Nm}$ with gearbox). This verifies that the motor specification is not a limiting factor for gentle indoor slopes, but rather the key factors are top speed on flat ground and battery life.

Buck Converter Stability

The custom MATLAB script (in Appendix 6) calculated the open-loop gain and phase of the compensated buck converter at 2000 logarithmically spaced frequencies from 10 Hz to 1 MHz. Stability margins are:

- **Crossover frequency:** $f_c = 3.2\text{ kHz}$
- **Phase margin:** $\text{PM} = 49.5^\circ$

- **Gain margin:** $GM > 20$ dB (phase never crosses -180° while gain > 0 dB)

The crossover frequency (3.2 kHz) is well below the maximum recommended value of $f_s/10 = 34\text{ kHz}$, so the control loop will not try to compensate for disturbances at a faster rate than the 340 kHz switching period. The phase margin of 49.5° meets the minimum requirement of 45° and suggests that the system will have a stable, slightly underdamped transient response. Transient tests (simulated by changing the load resistance in the MATLAB model) show that the voltage overshoot is about 8% and the settling time is 0.3 ms, which is suitable for digital loads..

PID Performance

The PID heading controller was tested both in a MATLAB simulation and by actually driving the robot on the lab floor.

Simulation Results

The discrete simulation (Appendix 7) assumed the robot's rotational dynamics to be a first-order lag ($\tau=0.1$ s) controlled by the PD controller. With a 90-degree step input:

- **Rise time (10%–90%):** ≈ 0.15 s
- **Settling time ($\pm 2\%$):** ≈ 0.5 s
- **Overshoot:** None (overdamped response)
- **Steady-state error:** Zero (in the ideal linear model)

The lack of overshoot is due to a low K_p (0.03) and derivative damping ($K_d = 0.01$). Although higher K_p will reduce rise time, it will also lead to overshoot and instability to floor jitter. The parameter values are a conservative tuning, preferring a soft response to a more aggressive one, as jerky movements in a hospital could frighten people or knock over cargo.

Root-Locus Analysis

The root-locus of the closed-loop poles, as K_p varies from 0 to 2 with $K_d = 0.01$ fixed, is shown in Figure 3.5. Since the two poles are always on the negative real axis (left half-plane) for $K_p > 0$, the system is stable with any positive proportional gain. With the tuned gain $K_p = 0.03$, the dominant pole is $s_1 \approx -0.03$ (slow exponential mode) and the fast pole $s_2 \approx -10.07$ is due to the motor electrical time constant (quickly dies out). With real poles, the step response is overdamped, and there are no oscillations, as observed by the absence of overshoot.

Physical Driving Tests

The robot could drive in a straight line for more than 3 m on the laboratory floor (smooth vinyl tiles with occasional expansion joints). The size of the PID correction was usually less than 0.1 (10% of the maximum motor speed), which meant that the gyroscopy feedback was making small corrections instead of large turns. When a wheel hit a small bump in the floor, the yaw disturbance was detected by the gyroscope in less than 50 ms, and the PID controller recovered the desired heading in 0.3 s.

SLAM Mapping

The BreezySLAM algorithm was tested on 847 scans of a 2-minute manual drive around the robotics laboratory. This 6 m $\times$ 8 m lab has workbenches, chairs, a whiteboard and a doorway. The occupancy grid (Figure 3.6) properly outlines the room's rectangular boundaries, internal obstacles and the doorway.

Key observations from the map:

- Wall boundaries show thickness of 2-3 pixels (2-3 cm), matching the resolution (1 cm at 3 m) of the LiDAR.
- The door on the left is open (white free space) instead of closed.
- There are small black bunches of chair legs in the middle.
- The upper right corner of the floor is gray because the robot didn't go to that corner during the logging run.

Scan matching was able to correct odometry drift. Using the raw encoder-only poses (without SLAM correction) for projecting scans resulted in walls as triple lines due to accumulated error in heading of about 5° per revolution of the robot. This effect was eliminated by the corrected poses after the BreezySLAM scan matching, showing the success of the RMHC algorithm.

Autonomous Navigation

1. We validated the integrated autonomous navigation routine on a 4 m $\times$ 5 m room with no furniture and a single obstacle (cardboard box).
2. The procedure was: Drive around the area to create an occupancy grid map.
3. Specify the initial and final cells.
4. Then use the A* algorithm to find the shortest path. Instruct the robot to follow the waypoints.

The A* pathfinding algorithm found a 14-cell-length path around the obstacle in 0.04 s. The robot successfully passed all waypoints and arrived at the goal with an error of around 15 cm (one cell width). The largest error came from encoder drift during the 3 m dead-reckoning run; the lack of LiDAR-based localization during the run resulted in linear increase of the dead-reckoning error. With the addition of AMCL or periodic scan matching in future, this error will be reduced to less than 5 cm.

Performance Characterization

The performance of the overall autonomous navigation was evaluated by recording the time for each step of the mission. A map of the 847-scan CSV file took 12.3 seconds to process on the Raspberry Pi 5 (single core). The A* shortest path planning on the 40 $\times$ 40 downsampled map took an average of 38 ms, much shorter than real-time. Waypoint following at 0.3 m/s base speed led to a traversal speed of about 1.2 m/min (including the in-place turn delays). The average time

to complete a 20 m long hospital corridor route would be 17 minutes, which is comparable to human walking speed and sufficient for slow material transport.

The encoder drift rate was empirically determined by driving the robot in a straight line (3 m) ten times and measuring the lateral deviation at the end of the line. The result was an average absolute value of 8.3 cm, and a standard deviation of 3.1 cm, showing that dead reckoning based only on encoder data is sufficient for short distances but needs to be corrected for longer distances.

VNC Connection

Remote desktop connection was set-up from a Windows 10 laptop to the Raspberry Pi 5 over the local Wi-Fi network. We tested both methods of configuration:

Method 1 (X11): The Pi was switched to X11 mode and the RealVNC server started. The desktop resolution was at 1920\times1080 and the latency was low (<100 ms) on the local network. The desktop and all menus and applications (including the Pygame LiDAR visualizer) rendered properly.

Method 2 (Wayland): With wayvnc configured to use the RSA key, the client connected, after accepting the unencrypted-warning. The results were similar to Method 1 with minor distortion of some desktop effects (shadows) caused by the framebuffer compression.

Figure 3.9 shows the VNC connection showing the robot desktop. This feature allows for remote monitoring and teleoperation without a display, keyboard and mouse connected to the robot.

Conclusions and Future Work

Summary of Achievements

In this project, we have designed, constructed and tested a fully autonomous Unmanned Ground Vehicle (UGV) for force analysis, stable power supply and indoor navigation in real time with an off-the-shelf Raspberry Pi 5 computer running pure-Python software. Here are the project's highlights:

1. Force Analysis Simulator. We developed a dynamic simulator to calculate the six important force and torque components for inclinations. This proved that the chosen JGA25-370 motors and wheels with a radius of 0.038 m will have more than sufficient torque for hospital grade slopes ($< 5^\circ$), but will probably slip on average flooring for slopes over 35° .

2. Stable Buck Converter. A synchronous buck converter was also designed using the MSMP2307DN-LF-Z, with complete analytical development of the transfer functions of the plant and compensator. Type-II compensation ($R_{12} = 6.8 \text{ k}\Omega$, $C_{12} = 3.9 \text{ nF}$) was verified by custom scripts (no Control System Toolbox) in MATLAB to have a crossover frequency of 3.2 kHz and a phase margin of 49.5° , exceeding the 45° stability criterion.

3. Encoder Odometry and PID Control. Hall-effect encoders had 47 ticks per revolution (5.08 mm per tick). Differential kinematics were used to calculate pose (x,y, θ) in Python at 10 Hz. A discrete-time PD controller with $K_p = 0.03$, $K_i = 0$, and $K_d = 0.01$ maintained heading errors of straight-line trajectories at $\pm 2^\circ$ or less on level surfaces. The MATLAB step response and root-locus revealed the system was stable with real negative poles at $s_1 \approx -0.03$ and $s_2 \approx -10.07$.

4. SLAM and Occupancy Mapping. A total of 847 LiDAR scans (RPLidar A1) and the encoder odometry were processed using BreezySLAM to generate a 800×800 pixel occupancy map of the lab. The LiDAR scans were scan matched to remove odometry drift, producing sharp edges on the walls, rather than rotational blurring with dithered dead reckoning.

5. A* Path Planning and Autonomous Execution. The A search algorithm computed optimal paths on the occupancy grid in just a few milliseconds. The robot was able to follow out path in a loop to avoid an obstacle with 14-waypoints, achieving a goal precision of around 15 cm, performing dead reckoning with encoders only.

6. Remote VNC Access. VNC Access was enabled on both X11 and Wayland, allowing remote wireless desktop access for monitoring and debugging, as well as manual control of the robot without having peripherals connected to the robot. These results demonstrate the ability to effectively provide the basic functions required for autonomous indoor navigation (mapping, localization, planning and control) for a low cost, open-source, stand-alone Python implementation, without the need for ROS or expensive, dedicated hardware.

Lessons Learned

There are a couple of lessons learned in the design process that should be noted. First, motor and logic power supplies should be electrically isolated; some of the first robots that used the same 5 V regulator to power the motors and the Pi sometimes reset the Pi when the motor direction was

changed. Second, the ZK-5AD motor driver does not have separate enable pins for motors so motor speeds have to be controlled in software (which was not a problem after discovering that the gpiozero library automatically does PWM). Third, the RMHC scan matcher in the BreezySLAM library is sensitive to regular scans; the Raspberry Pi's CPU is heavily burdened by Pygame rendering, and occasional loss of scans caused temporary errors in the map which were later corrected. This would be improved by moving SLAM to a separate thread or process.

Future Work

While the present system achieves the project objectives, there are numerous things that could be improved:

Take into Account Rolling Resistance and Dynamic Forces. Rolling resistance, aerodynamic and acceleration forces are not included in the force simulator. Inclusion of the coefficient of rolling resistance C_{rr} (typically 0.01-0.02 for rubber on concrete floor) and the inertia of wheels would give better torque estimates, particularly during dynamic situations like starting from rest, or braking.

Hardware Implementation of Buck Converter. In this project, a buck converter was designed and simulated; the next phase should be to design and build the printed circuit board (PCB), mounting the components and measure its efficiency, ripple and transient response to a step change in load (using an electronic load and oscilloscope).

Adaptive Monte Carlo Localization (AMCL). BreezySLAM enables scan matching during the mapping step, but during the navigation step, the robot only uses odometry. AMCL or any other module based on a particle filter would allow the robot to follow the planned path and re-estimate its pose by using the current LiDAR scans and the map built in the mapping phase, thus reducing the current 15 cm pose error at the end of the navigation phase to less than 5 cm.

Dynamic Obstacle Avoidance. The existing obstacle-avoidance algorithm halts the robot in case there's an unexpected obstacle in the LiDAR front. An improved local planner (such as Dynamic Window Approach (DWA), Vector Field Histogram (VFH) etc.) would generate a detour path to avoid dynamic obstacles (pedestrians) or temporary obstacles (trash bins) and get back to the A* planned path.

Elevator Support for Inter-Floor. To support hospital navigation, the UGV has to navigate elevators. This includes Wi-Fi-based floor detection (different Wi-Fi SSIDs or signal strength) and switching the occupancy map (to the relevant floor) and pushing the elevator buttons (servo or relay).

Payload Integration and Docking. For the medical delivery use case, a secure storage for the payload and reading RFID or barcodes are necessary to deliver the correct medication to the correct ward. Power and data ports for remote charging and communication would increase the UGV's operation time and allow a fleet of UGVs.

Machine Learning for Scene Understanding. A camera (e.g., Raspberry Pi Camera Module 3) and low-weight neural networks (MobileNet, YOLO-Lite) would detect obstacles (person, chair, door, wall) enabling context-aware operation such as slowing down when encountering a person, and speaking at the destination using text-to-speech.

Power System Optimization. The current battery (LiPo 2S) has low run-time. A power-management approach to intelligently reduce the speed of the motor and LiDAR scan if no action is needed, and boost-buck converter to handle a wider range of input voltage, would boost efficiency and run-time.

To sum up, this project provides a strong basis for low-cost autonomous indoor server robots. By carefully addressing the mechanical force requirement, the electric power stability, and the software-based navigation intelligence with an open-source platform, it shows that a flexible mobile robot behaviour can be achieved with limited resources that directly contribute to the inclusion of innovation and sustainable urban infrastructure as per the objective 9 and 11 of SDGs.

REFERENCES

- [1] S. Levy, "BreezySLAM: Simple, efficient SLAM in Python," *GitHub Repository*, 2020. [Online]. Available: <https://github.com/simondlevy/BreezySLAM>
- [2] M. Monette, "MP2307: 3A, 23V, 340KHz Synchronous Rectified Step-Down Converter," *Monolithic Power Systems Datasheet*, Rev. 1.15, 2019.
- [3] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.
- [4] J. Borenstein and L. Feng, "Measurement and correction of systematic odometry errors in mobile robots," *IEEE Transactions on Robotics and Automation*, vol. 12, no. 6, pp. 869–880, Dec. 1996.
- [5] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, vol. 3, 2004, pp. 2149–2154.
- [6] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.
- [7] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, Jul. 1968.
- [8] Raspberry Pi Foundation, "Raspberry Pi 5 Documentation," 2024. [Online]. Available: <https://www.raspberrypi.com/documentation/computers/raspberry-pi-5.html>
- [9] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, Nov. 2000.
- [10] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
- [11] R. C. Dorf and R. H. Bishop, *Modern Control Systems*, 13th ed. Boston, MA, USA: Pearson, 2017.
- [12] InvenSense, "MPU-6000 and MPU-6050 Product Specification Revision 3.4," *InvenSense Datasheet*, 2013.
- [13] Slamtec, "RPLidar A1 Low Cost 360 Degree 2D Laser Scanner," *Slamtec Product Datasheet*, 2021.
- [14] RealVNC Ltd., "RealVNC Connect Documentation," 2024. [Online]. Available: <https://help.realvnc.com>

MATLAB Code for Buck Converter Bode Plot

The following MATLAB script computes the open-loop transfer function of the buck converter manually (without the Control System Toolbox) and plots the Bode diagram while reporting the crossover frequency and phase margin.

```
% Buck Converter Stability Analysis for Defense (Base MATLAB)
clear; clc; close all;
% 1. System Parameters
Vout = 5.0;           % Output Voltage (V)
Iout = 5.0;           % Max Output Current (A)
R_load = Vout / Iout; % Effective Load Resistance (Ohms) C_out =
470 e - 6;           % Output Capacitance (F)
ESR = 30 e - 3;       % Estimated Output Capacitor ESR (Ohms)
% IC Internal Parameters (Typical for MP2307/Current Mode) Vref = 0.9
25;                  % Feedback reference voltage (V)
gm_ea = 1000 e - 6;   % Error Amplifier Transconductance (A/V) G_cs = 3.3
3;                   % Current Sense Transconductance (A/V)
% 2. Compensator Parameters
R12 = 6800;           % Compensation Resistor (Ohms) C12 = 3.
9 e - 9;              % Compensation Capacitor (F)
% 3. Frequency Vector Setup f = logs
pace(1, 6, 2000);
w = 2 * pi * f; s = 1j *
w;
G_plant = (R_load * G_cs) .* (Plant_zero ./ Plant_pole);
Z_comp = R12 + (1 ./ (s .* C12)); Feedback_Gain = Vref /
Vout;
G_comp = Feedback_Gain .* gm_ea .* Z_comp;
G_open_loop = G_comp .* G_plant;

% 5. Extract Magnitude and Phase
Mag_linear = abs(G_open_loop);
Mag_dB = 20 * log10(Mag_linear);
Phase_rad = unwrap(angle(G_open_loop));
Phase_deg = Phase_rad * (180 / pi);
41
42 % 6. Calculate Phase Margin and Crossover Frequency
43 [~, crossover_idx] = min(abs(Mag_dB));
44 fc = f(crossover_idx);
45 pm = Phase_deg(crossover_idx) + 180;
47 disp('--- Stability Margins (Manual Calculation)    ---');
48 fprintf('Crossover Frequency (fc): %.2f kHz\n', fc / 1000);
```



```

49 fprintf('Phase Margin (PM): %.2f degrees\n', pm);
50
51 % 7. Plot Custom Bode Plot
52 figure('Name', 'Open Loop Bode Plot', 'Color', 'w');
53
54 subplot(2,1,1);
55 semilogx(f, Mag_dB, 'b', 'LineWidth', 2); hold on;
56 plot([min(f) max(f)], [0 0], 'k--', 'LineWidth', 1);
57 plot(fc, Mag_dB(crossover_idx), 'ro', 'Marker Size', 8,
    'Marker Face Color', 'r');
58 grid on;
59 title('Open-Loop Bode Plot (Magnitude)');
60 ylabel('Magnitude (dB)');
61 xlim([min(f) max(f)]);
62
63 subplot(2,1,2);
64 semilogx(f, Phase_deg, 'b', 'LineWidth', 2); hold on;
65 plot([min(f) max(f)], [-180 -180], 'k--', 'LineWidth', 1);
66 plot(fc, Phase_deg(crossover_idx), 'ro', 'Marker Size', 8,
    'Marker Face Color', 'r');
67 grid on;
68 title('Open-Loop Bode Plot (Phase)');
69 ylabel('Phase (Degrees)');
70 xlabel('Frequency (Hz)');
71 xlim([min(f) max(f)]);

```

```

sgtitle(sprintf('Stability: PM = %.1f deg at %.1f kHz', pm, fc
    /1000), ...
    'FontSize', 14, 'FontWeight', 'bold');

```

Listing A.1: Buck Converter Stability Analysis (Base MATLAB)

Appendix B

MATLAB Code for PID Step Response and Root Locus

The following MATLAB script simulates the discrete-time PID controller using manual Euler integration, plots the step response, and computes the root locus by sweeping K_p .

```
1
2
3 % Raw Discrete PID Simulation (No Control System Toolbox)
4 clear; clc; close all;
5
6 % 1. Simulation Setup dt
7 dt = 0.01; % Time step (seconds)
8 t_end = 5; % Total simulation time (seconds)
9
10 t = 0:dt:t_end; N =
11 length(t);
12
13 % 2. Robot Physics (Plant Parameters)
14 tau = 0.1; % Motor time constant
15 % 3. Your Tuned PID Parameters Kp = 0.03;
16 Ki = 0.00;
17 Kd = 0.01;
18 theta_target = 90;
19 theta = zeros(1, N); omega = zeros(1, N); u =
20 zeros(1, N);
21 error = zeros(1, N);
22 integral_err = 0; prev_err = 0;
23 prev_err = error(k);
24
25 angular_acceleration = (u(k) - omega(k)) / tau; omega(k+1) = omega(k)
26 + (angular_acceleration * dt); theta(k+1) = theta(k) + (omega(k)
27 * dt);
28 error(N) = theta_target - theta(N);
29
30
3
3
```

```

41 u(N) = u(N-1);
42
43 % 5. PLOT THE RESULTS
44 figure('Name', 'Raw PID Step Response ', 'Color ', 'w');
45
46 subplot(2,1,1);
47 plot(t, theta, 'b-', 'LineWidth', 2); hold on;
48 plot(t, theta_target * ones(1,N), 'r--', 'LineWidth', 1.5);
49 title('UGV Heading Correction (Kp=0.03, Kd=0.01)');
50 xlabel('Time (s)'); ylabel('Heading Angle (Degrees)');
51 legend('Actual Heading', 'Target (90 deg)', 'Location', 'southeast')
    ;
52 grid on;
53
54 subplot(2,1,2);
55 plot(t, u, 'g-', 'LineWidth', 1.5);
56 title('Control Effort (Motor Adjustment)');
57 xlabel('Time (s)'); ylabel('PID Output Signal');
58 grid on;
59
60 % 6. ROOT LOCUS (Manual Sweep)
61 Kd = 0.01;
62 Kp_values = 0:0.001:2;
63 roots_real = zeros(length(Kp_values), 2);
64 roots_imag = zeros(length(Kp_values), 2);
65
66 for i = 1:length(Kp_values)
67     Kp = Kp_values(i);
68     coeffs = [tau, (1 + Kd), Kp];
69     r = roots(coeffs);
70     roots_real(i, 1) = real(r(1)); roots_imag(i, 1) = imag(r(1));
71     roots_real(i, 2) = real(r(2)); roots_imag(i, 2) = imag(r(2));
72 end

```

```
figure('Name', 'Raw Root Locus ', 'Color ', 'w');
plot(roots_real(:,1), roots_imag(:,1), on; 'b-', 'LineWidth', 2); hold
plot(roots_real(:,2), roots_imag(:,2), plot(roo
ts_real(1,:), roots_imag(1,:),
'LineWidth', 2);
'kx', 'MarkerSize', 10, '

```

79

```
tuned_idx = find(abs(Kp_values - 0.03) < 0.0001, 1);
plot(roots_real(tuned_idx,:), roots_imag(tuned_idx,:), 'go', ... 'Marker Size', 8
, 'LineWidth', 2, 'Marker Face Color', 'g');

title(sprintf('Root Locus (Fixed Kd = %g)', Kd));
xlabel('Real Axis (sigma)'); ylabel('Imaginary Axis (jomega)'); grid on; axis
equal;
legend('Pole 1', 'Pole 2', 'Open-Loop (Kp=0)', 'Tuned Kp=0.03');
```

Appendix C

Python Data Logger for SLAM

The following excerpt from `map_logger.py` shows the core SLAM data-logging loop that synchronizes odometry, gyroscope, and LiDAR data into a CSV file.

```
1 # Inside the main loop of map_logger.py
for new_scan, quality, angle, distance in lidar.iter_measurments():
2     :
3     current_time = time.time()
4     # Fast control loop (20 Hz)
5     current_time - last_motor_time >= 0.05: dt = current_time - last_motor_time
6     last_motor_time = current_time

7     # Read Gyro
8     gyro_data = sensor.get_gyro_data() current_yaw += gyro_data['z'] * dt

9     # Handle keyboard input (teleoperation) # ... (event handling code) ...
10    # Apply PID steer assist if driving straight:
11    adjustment = pid.compute(target_yaw, current_yaw, dt) left_speed = max(0.0,
12    min(1.0,
13        (BASE_SPEED * FWD_LEFT_TRIM) - adjustment)) right_speed = max(0.0, min(1.
14    0,
15        (BASE_SPEED * FWD_RIGHT_TRIM) + adjustment)) left_motor.forward(left_spe
16    ed)
17    right_motor.forward(right_speed)
18    # Update Odometry
19    current_left_ticks = left_encoder.steps
20    current_right_ticks = right_encoder.steps

21    left = (current_left_ticks - prev_left_ticks) *
22    METERS_PER_TICK

23    d_right = (current_right_ticks - prev_right_ticks) *
```

```
METERS_PER_TICK
```

```
d_center = (d_left + d_right) / 2.0
```

```
d_theta = (d_right - d_left) / WHEEL_BASE_M
```

```
robot_x += d_center * math.cos(robot_theta)robot_y += d_center * math.sin(robot_theta)robot_theta += d_theta
```

```
prev_left_ticks = current_left_ticksprev_right_ticks = current_right_ticks
```

```
# Slow LiDAR processing (per scan)
```

```
quality > 10 and 200 <= distance <= 3000:angle_idx = min(359, int(angle))
```

```
scan_distances[angle_idx] = int(distance / 10)
```

```
if new_scan:
```

```
csv_line = f"{robot_x:.4f},{robot_y:.4f},{current_yaw:.2f}\n"
```

```
log_file.write(csv_line)
```

```
scans_recorded += 1scan_distances = [0] * 360
```

APPENDIX D — UGV TUNING MANUAL FOR CREW MEMBERS

This appendix provides a concise, jargon-free tuning guide for team members who need to operate or maintain the UGV without detailed knowledge of the underlying code. All tuning variables are located in the `teleop_tracker.py` and `autonomous_navigator.py` scripts under the SPEED & CALIBRATION and PID CONTROLLER sections.

Overall Power: The Speed Limit

Variable: `BASE_SPEED` (range 0.0 to 1.0). This acts as the master throttle. If the robot pops a wheelie or accelerates too violently, lower it to 0.4. If the robot struggles to climb a bump, raise it to 0.7.

Manual Trim: Fixing Natural Drift

Variables: `FWD_LEFT_TRIM`, `FWD_RIGHT_TRIM`, `BWD_LEFT_TRIM`, `BWD_RIGHT_TRIM` (range 0.0 to 1.0). No two motors are identical; one will always push harder, causing the robot to curve. Use trim to throttle the faster motor down.

To tune forward trim: Temporarily disable auto-steering by setting `Kp = 0.0` in the PID line. Drive forward. If the robot curves right, the left motor is too strong; lower `FWD_LEFT_TRIM` (e.g., from 0.85 to 0.80). If it curves left, raise it slightly. Repeat until the robot drives straight.

To tune backward trim: Drive in reverse. If the rear curves right, the right motor is too strong; lower `BWD_RIGHT_TRIM`. Adjust in 0.05 increments.

The PID Controller: Auto-Steering

Variables: K_p , K_i , K_d . The PID controller acts like an algorithmic driver, using the MPU6050 gyroscope to detect drift and automatically steer back to the target heading.

K_p (Proportional): The immediate reaction force. If K_p is too low, the robot drifts without fighting back. If K_p is too high, the robot zig-zags violently. Start at 0.03 and adjust by ± 0.01 .

K_d (Derivative): The shock absorber. Increase K_d (e.g., from 0.01 to 0.03) if the robot corrects its path but wiggles at the end of the correction. Decrease K_d if the robot feels sluggish to react.

K_i (Integral): Leave at 0.0 unless a heavy, off-centre payload constantly pulls the robot to one side. In that case, introduce a very small K_i (0.001) and monitor for windup.

C.1 Standard Tuning Sequence

Follow these steps in order whenever the robot is replaced on a new floor surface or after changing the battery or payload:

1. **Disable PID:** Set $K_p = 0.0$, $K_i = 0.0$, $K_d = 0.0$.
2. **Fix Forward:** Drive forward and adjust FWD trims until the path is straight.
3. **Fix Reverse:** Drive backward and adjust BWD trims until the path is straight.
4. **Enable PID:** Restore $K_p = 0.03$ and $K_d = 0.01$. Drive over a bumpy floor and adjust K_p and K_d until the robot fights bumps smoothly without jitter.
5. **Verify:** Conduct a 2-metre straight-line test and confirm lateral deviation is less than 3 cm.

Document any changed values in the project logbook so that previous settings can be restored if a change proves detrimental.

ORIGINALITY REPORT

7%

SIMILARITY INDEX

5%

INTERNET SOURCES

4%

PUBLICATIONS

4%

STUDENT PAPERS

PRIMARY SOURCES

1

Submitted to American Public University System

Student Paper

<1%

2

Submitted to University of Essex

Student Paper

<1%

3

Suryawanshi, Hrigved Mahesh. "Traversability Aware Autonomous Navigation for Multi-Modal Mobility Morphobot (M4)", Northeastern University, 2025

Publication

<1%

4

arxiv.org

Internet Source

<1%

5

Bogdan M. Wilamowski, J. David Irwin. "Control and Mechatronics - The Industrial Electronics Handbook", CRC Press, 2018

Publication

<1%

6

hal.science

Internet Source

<1%

7

Fundamentals of Power Electronics, 2001.

Publication

<1%

8

Submitted to Virginia Polytechnic Institute and State University

Student Paper

<1%

Submitted to LTI 202223

9	Student Paper	<1 %
10	Submitted to Monash University Student Paper	<1 %
11	caeug.net Internet Source	<1 %
12	123dok.net Internet Source	<1 %
13	www.coursehero.com Internet Source	<1 %
14	Submitted to Babes-Bolyai University Student Paper	<1 %
15	Submitted to Columbia University Student Paper	<1 %
16	www.termpaperwarehouse.com Internet Source	<1 %
17	M.L. Nagurka. "Gain and phase margins of SISO systems from modified root locus plots", IEEE Control Systems Magazine, 06/1992 Publication	<1 %
18	Chen Chen, Liang Wang, Wanyang Wang, Run Min, Qiaoling Tong. "Online Inductance Monitoring Based on Dynamic Characteristics and ESR Effect Compensation for Buck Converter Without Current Sensor", Sensors, 2025 Publication	<1 %
19	Submitted to Midlands State University Student Paper	<1 %

20	Submitted to University of Westminster Student Paper	<1 %
21	Submitted to European Universities in Egypt Student Paper	<1 %
22	python.hotexamples.com Internet Source	<1 %
23	econpapers.repec.org Internet Source	<1 %
24	Poblete, Pablo. "Advanced Power Control System for Enhancing Second-Life Battery Energy Storage Systems Integration and Reliability.", Pontificia Universidad Catolica de Chile (Chile) Publication	<1 %
25	anpl-technion.github.io Internet Source	<1 %
26	sinay.ai Internet Source	<1 %
27	www.allaboutcircuits.com Internet Source	<1 %
28	www.mdpi.com Internet Source	<1 %
29	Advances in Intelligent Systems and Computing, 2014. Publication	<1 %
30	Submitted to University of Newcastle upon Tyne Student Paper	<1 %
31	Submitted to Purdue University	

<1 %

32

Robert M. Koretsky. "Raspberry Pi 5 System Administration Basics", CRC Press, 2025

Publication

<1 %

33

Submitted to University of Southampton

Student Paper

<1 %

34

www.parlament.gv.at

Internet Source

<1 %

35

Lorenzo Scalera, Eleonora Maset, Diego Tiozzo Fasiolo, Khalid Bourr et al. "Forest Surveying with Robotics and AI: SLAM-Based Mapping, Terrain-Aware Navigation, and Tree Parameter Estimation", Machines, 2026

Publication

<1 %

36

Submitted to The University of Texas at Arlington

Student Paper

<1 %

37

Submitted to University of Greenwich

Student Paper

<1 %

38

Submitted to University of Hertfordshire

Student Paper

<1 %

39

manglar.uninorte.edu.co

Internet Source

<1 %

40

Submitted to Pembroke School

Student Paper

<1 %

41

www.datasheetbank.com

Internet Source

<1 %

42	Mazen, Amna. "Trajectory Optimization of Wheeled Mobile Robots Based on Bézier Curves for Online Navigation.", University of Detroit Mercy Publication	<1 %
43	Williams, Terrance Rodriquez, Jr.. "Leader-Follower-Based Collaborative Navigation of a UGV-UAV Formation", The University of West Florida, 2024 Publication	<1 %
44	core-electronics.com.au Internet Source	<1 %
45	publica-rest.fraunhofer.de Internet Source	<1 %
46	www2.mdpi.com Internet Source	<1 %
47	Fundamentals of Power Electronics, 1997. Publication	<1 %
48	Yi Liu, Rongwu Zhu, Yang Leng, Marco Liserre. "Transient Performances Analysis of Talkative Dual-Active Bridge Converter Modulated by PWM-FSK", IECON 2023- 49th Annual Conference of the IEEE Industrial Electronics Society, 2023 Publication	<1 %
49	mail.ijcrt.org Internet Source	<1 %
50	Submitted to Cornell University Student Paper	<1 %

51	Design and Management of Energy-Efficient Hybrid Electrical Energy Storage Systems, 2014. Publication	<1 %
52	Shrivastava, Ashish, and Bhim Singh. "Buck converter-based power supply design for low power light emitting diode lamp lighting", IET Power Electronics, 2014. Publication	<1 %
53	bucket-download.slamtec.com Internet Source	<1 %
54	ebin.pub Internet Source	<1 %
55	gill.readingroo.ms Internet Source	<1 %
56	iksor.si Internet Source	<1 %
57	pdf.datasheet.live Internet Source	<1 %
58	psasir.upm.edu.my Internet Source	<1 %
59	pypi.org Internet Source	<1 %
60	res.mdpi.com Internet Source	<1 %
61	slidelegend.com Internet Source	<1 %
62	sp.ltc.gov.on.ca Internet Source	<1 %

63	Ismail Keshta. "A cloud-assisted key agreement protocol for the E-healthcare system", PLOS One, 2025 Publication	<1 %
64	Rajalakshmi Pachamuthu, Bhaskar Anand, Abhishek Thakur, Parvez Alam. "LiDAR Technology for Intelligent Transportation and Autonomous Systems", CRC Press, 2025 Publication	<1 %
65	William S. Levine. "Control System Fundamentals", CRC Press, 2019 Publication	<1 %
66	archive.org Internet Source	<1 %
67	macsphere.mcmaster.ca Internet Source	<1 %
68	people.csail.mit.edu Internet Source	<1 %
69	rawatdigital.in Internet Source	<1 %
70	www.trinityos.com Internet Source	<1 %
71	M. R. D. AL-MOTHAFAR. "Average current-mode control of a two-module DC-DC converter with mutually coupled output filter inductors", International Journal of Electronics, 2010 Publication	<1 %
72	digitalcommons.calpoly.edu Internet Source	<1 %

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	Off		

FINAL GRADE

GENERAL COMMENTS

/0

PAGE 1

PAGE 2

PAGE 3

PAGE 4

PAGE 5

PAGE 6

PAGE 7

PAGE 8

PAGE 9

PAGE 10

PAGE 11

PAGE 12

PAGE 13

PAGE 14

PAGE 15

PAGE 16

PAGE 17

PAGE 18

PAGE 19

PAGE 20

PAGE 21

PAGE 22

PAGE 23

PAGE 24

PAGE 25

PAGE 26

PAGE 27

PAGE 28

PAGE 29

PAGE 30

PAGE 31

PAGE 32

PAGE 33

PAGE 34

PAGE 35

PAGE 36

PAGE 37

PAGE 38

PAGE 39

PAGE 40

PAGE 41

PAGE 42

PAGE 43

PAGE 44

PAGE 45

PAGE 46

PAGE 47

PAGE 48

PAGE 49

PAGE 50

PAGE 51

PAGE 52

PAGE 53

PAGE 54

PAGE 55

PAGE 56

PAGE 57

PAGE 58

PAGE 59

PAGE 60

PAGE 61

PAGE 62

PAGE 63

PAGE 64
